

Pitfalls in Language Models for Code Intelligence: A Taxonomy and Survey

XINYU SHE, Huazhong University of Science and Technology, Wuhan, China

YUE LIU, Monash University, Melbourne, Australia

YANJIE ZHAO, Huazhong University of Science and Technology, Wuhan, China

YILING HE, Zhejiang University, Hangzhou, China

LI LI, Beihang University, Beijing, China

CHAKKRIT TANTITHAMTHAVORN, Information Technology, Monash University, Clayton, Australia

ZHAN QIN, Zhejiang University, Hangzhou, China

HAOYU WANG, Huazhong University of Science and Technology, Wuhan, China

Modern Language Models (LMs) have been successfully employed in source code generation and understanding, leading to a significant increase in research focused on learning-based code intelligence, such as automated bug repair and test case generation. Despite their great potential, *LMs for code intelligence (LM4Code) are susceptible to potential pitfalls, which hinder realistic performance and further impact their reliability and applicability in real-world deployment*. Such challenges drive the need for a comprehensive understanding—not just identifying these issues but delving into their possible implications and existing solutions to build more reliable LMs tailored to code intelligence. Based on a well-defined systematic research approach, we conducted an extensive literature review to uncover the pitfalls inherent in LM4Code. Finally, 121 primary studies from top-tier venues have been identified. After carefully examining these studies, we designed a taxonomy of pitfalls in LM4Code research and conducted a systematic study to summarize the issues, current solutions, implications, and challenges of different pitfalls for LM4Code systems. We developed a comprehensive classification scheme that dissects pitfalls across four crucial aspects: data collection and labeling, system design and learning, performance evaluation, and deployment and maintenance. Through this study, we aim to provide a roadmap for researchers and practitioners, facilitating their understanding and utilization of LM4Code in reliable and trustworthy ways.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Artificial intelligence**;

This work was supported by the Key R&D Program of Hubei Province (2023BAB079, 2023BAB017) and the National Key Laboratory of Data Space Technology and System.

Xinyu She and Yue Liu are co-first authors who contributed equally to this work.

Authors' Contact Information: Xinyu She, Huazhong University of Science and Technology, Wuhan, China; e-mail: xinyushe@hust.edu.cn; Yue Liu, Monash University, Melbourne, Australia; e-mail: yue.liu1@monash.edu; Yanjie Zhao, Huazhong University of Science and Technology, Wuhan, China; e-mail: carolzhao233@gmail.com; Yiling He, Zhejiang University, Hangzhou, China; e-mail: yilinghe@zju.edu.cn; Li Li, Beihang University, Beijing, China; e-mail: lilicoding@ieee.org; Chakkrit Tantithamthavorn, Information Technology, Monash University, Clayton, Australia; e-mail: chakkrit@monash.edu; Zhan Qin, Zhejiang University, Hangzhou, China; e-mail: qinzhan@zju.edu.cn; Haoyu Wang (corresponding author), Huazhong University of Science and Technology, Wuhan, China; e-mail: haoyuwang@hust.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://doi.org/10.1145/3748647).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART86

<https://doi.org/10.1145/3748647>

Additional Key Words and Phrases: Language models for Code, Software engineering, Code generation, Code intelligence, Trustworthiness

ACM Reference format:

Xinyu She, Yue Liu, Yanjie Zhao, Yiling He, Li Li, Chakkrit Tantithamthavorn, Zhan Qin, and Haoyu Wang. 2026. Pitfalls in Language Models for Code Intelligence: A Taxonomy and Survey. *ACM Trans. Softw. Eng. Methodol.* 35, 3, Article 86 (February 2026), 42 pages.
<https://doi.org/10.1145/3748647>

1 Introduction

With every upgrade, **Language Models (LMs)** seem to redefine future boundaries. LMs have achieved remarkable successes in natural language understanding and generation [103, 157], underlined by the significant contributions from state-of-the-art models such as T5 [37, 169], BERT [40, 42, 91], and GPT [81, 175]. Due to the format similarity between source code and natural language, LMs have been widely applied in the domain of **software engineering (SE)** [20, 109]. They are now extensively researched and employed for source code understanding and generation, such as code completion [120, 132], code summarization [42], code generation [88, 137], code search [147], program repair [32, 175], and test case generation [194]. With powerful learning capabilities, LMs have shown superior performance against traditional code intelligence approaches, such as template-based, heuristic-based, and machine learning-based approaches [62, 117, 132]. Their superior performance stems from the fact that many LMs are trained on vast and diverse code repositories, enabling LMs to discern complex syntax, comprehend semantic context, and effectively predict code sequences [161].

However, the lack of transparency, often termed “black-box,” poses significant challenges and concerns [150, 152]. In other words, while LMs for code intelligence (LM4Code) approaches offer powerful capabilities, they often lack transparency in their underlying reasoning and decision-making process. Jiarapakdee et al. [65] and Tantithamthavorn and Jiarapakdee [152] also raised concerns that such a lack of transparency often leads to a lack of adoption of LM4Code in practice. Consequently, hidden or neglected pitfalls in data or algorithms may persist, leading to unrealistic performance evaluation and unreliable code recommendations [55, 151]. For example, Shi et al. [142] found that noisy data (e.g., empty methods or duplicated code) was prevalent in widely used benchmark datasets for code summarization, with contamination levels ranging between 31% and 66%. By filtering out this noisy data, performance metrics like the BLEU-4 score witnessed a substantial increase (e.g., from 11.36% to 16.48%). Similarly, Sun et al. [147] highlighted a substantial amount of noise in user queries across various code search benchmark datasets. Such instances underscore the hidden data noise that might undermine the trustworthiness of code produced or recommended by LMs. What is more concerning is when these pitfalls go unnoticed, which raises significant questions about the reliability and integrity of the LM4Code systems built on them, thereby preventing the adoption of research advances in academia and industry.

As LMs become increasingly prevalent in code intelligence despite increasing obstacles, there emerges an urgent need for a comprehensive understanding of potential pitfalls within LM4Code systems. This is not limited to pitfall identification; it demands a deeper exploration into the understanding of the implications of these pitfalls, current solutions, and possible challenges. Although there is a growing body of research concerning or addressing pitfalls in LM4Code [96, 142, 147, 189], the domain lacks a comprehensive and systematic overview of these efforts. Without such an overview, researchers, developers, and practitioners potentially overlook significant pitfalls identified in previous studies. In this study, we conducted a **Systematic Literature Review (SLR)**,

adhering to a well-defined approach that identifies, evaluates, and interprets the relevant literature focusing on the pitfalls within LM4Code. Our contributions of this article are as follows:

- *Paper Collection of Pitfalls in LM4Code.* Through a rigorous SLR protocol as outlined by [70, 71] and after an in-depth analysis of the primary studies, we collected 121 primary papers (spanning 2018–2024) closely related to evaluating or addressing LM4Code pitfalls. Comprehensive details on our review process and the collected papers are available online.¹
- *Comprehensive Taxonomy.* We conducted a qualitative and quantitative synthesis of the collected studies. We present a taxonomy of the collected studies according to the LM4Code lifecycle, including data collection and labeling, system design and learning, performance evaluation, deployment, and maintenance. Our synthesis investigates the pitfalls present in LM4Code, summarizes the implications of these pitfalls, investigates how these issues are addressed, and outlines future challenges in this field.
- *Insightful Findings and Recommendations.* In addition to identifying and analyzing pitfalls, we distilled practical insights and recommendations for researchers and practitioners in the field of LM4Code. These findings pave the way for developing more robust and reliable LMs tailored for code intelligence, mitigating potential challenges, and maximizing their utility of such models in real-world applications.

2 Study Design

2.1 Research Questions (RQs) and Motivations

In recent research, LMs trained for code intelligence have shown promising performance [54, 164, 189]. However, an increasing number of literature [107, 142, 147] has highlighted the existence of pitfalls in LM4Code that can skew their realistic performance, leading to either substantial overestimation or underestimation of their effectiveness. The aim of conducting this systematic review is to gain an in-depth understanding of the pitfalls present in LMs tailored for code intelligence. Unlike surveys focusing on general LLMs or LM4Code [54, 165, 179, 185] that primarily catalog models, tasks, and evaluation metrics, our systematic review explicitly focuses on identifying and categorizing the pitfalls, solutions, and impacts associated with LM4Code in practice. While related research evaluating LM4Code [16, 92, 107, 176] identifies some issues, our work distinguishes itself in two key ways: (1) we employ a rigorous SLR methodology that systematically identifies and categorizes pitfalls, and (2) we present a comprehensive taxonomy structured around the entire LLM lifecycle, from dataset construction to deployment. This approach enables us to systematically analyze not only the identified issues but also their corresponding solutions and practical impacts, providing an integrative perspective that supports both researchers and practitioners in developing more reliable code intelligence systems.

Ensuring the robustness, reliability, and trustworthy deployment of LMs is important for their effective integration into the software development lifecycle. Consequently, it is crucial to discern the nature of these pitfalls, comprehend their implications, and examine existing solutions. Thus, we aim to answer the following RQs in this study:

- *RQ1: What types of pitfalls are prevalent in LMs for code intelligence?* This RQ aims to identify the prevalent pitfalls in LM4Code systems, exploring how they could affect various stages of the learning-based system lifecycle.
- *RQ2: What solutions have been proposed to address these pitfalls?* This RQ reviews the existing body of literature to identify proposed approaches for solving the identified pitfalls.

¹<https://github.com/yueyueL/ReliableLM4Code>.

- *RQ3: What are the implications of these pitfalls?* This RQ investigates the implications of the identified pitfalls, specifically focusing on their impacts on the effectiveness, reliability, and ethical considerations of LM4Code systems.

2.2 Data Collection and Extraction

To systematically identify relevant studies on LM4Code pitfalls, we conducted a lightweight SLR following the rigorous methodology proposed by Kitchenham and Charters [70], Kitchenham et al. [71], and Zhang et al. [192]. We employed the “**Quasi-Gold Standard**” (QGS) [192] approach, which combines manual and automated search strategies to ensure comprehensive coverage while maintaining high-quality standards. Our search encompassed major academic databases including ACM Digital Library, IEEE Xplore, Springer, ScienceDirect, Web of Science, and DBLP. To ensure the quality and relevance of selected papers, we implemented robust inclusion/exclusion criteria and conducted thorough **Quality Assessments (QAs)** of full texts.

During the review process, the first two authors independently examined and extracted information from each paper, developing an initial classification scheme for pitfalls based on thematic analysis. Guided by prior literature on machine learning system workflows [7, 54], we structured our taxonomy around four key stages: data collection and labeling, system design and learning, performance evaluation, and deployment and maintenance, as illustrated in Figure 3. Categorization disagreements were resolved through structured discussions among all authors until consensus was reached. A similar process guided the development of implication categories in Section 7. Additionally, we also conducted snowballing backward and forward [] to complement database searches and avoid excluding important works. Through this systematic combination of search strategies, selection criteria, and quality checks, we identified 121 high-quality studies investigating the pitfalls and challenges of LM4Code. The complete review protocol, including detailed search strategies, selection criteria, and QA procedures, is available in Appendix A.

2.3 Statistics of Collected Publications

Figure 1 displays the distribution of the collected research studies across the published year and published venues. From Figure 1(a), we have noted that there is a significant increase in the number of relevant research studies published annually from 2021. Prior studies, such as Watson et al. [170] and Yang et al. [181], have acknowledged the prevalence of LMs in code-related tasks between 2014 and 2020. However, our results indicate that limited attention has been paid to the identification and analysis of pitfalls in LM4Code. Nevertheless, Figure 1(a) shows a rising trend over the last 3 years, indicating an increasing interest in research within the research community about the potential pitfalls in LM4Code. Figure 1(b) shows the distribution of publications across SE, Security (SEC), and AI domains. The majority of papers come from the SE domain, particularly from top-tier venues such as ICSE and ASE. We also identified notable contributions from leading SEC and AI conferences, including four papers from S&P and six from ACL.

Figure 2 further presents the distribution of LMs used in the collected paper, and we arrange them in descending order as they appear from top to bottom. It is important to note that while both LSTM and GRU are types of RNN, in this study, articles that only specify the use of RNN without further detail are categorized under “General RNN.” Similarly, despite observing the utilization of several popular transformer-based architectures such as CodeBERT, CodeX, and CodeT5, papers that merely claim the use of a self-defined or custom-designed transformer are classified as “General Transformer” in subsequent sections. The temporal analysis in Figure 2 reveals LSTM’s dominance before 2020, followed by a significant shift toward transformer-based LMs in the past 4 years, particularly pre-trained models like CodeBERT and Codex. Recent years have seen increased investigation into **Large Language Models (LLMs)** such as GPT-4 and CodeLlama, aligning with

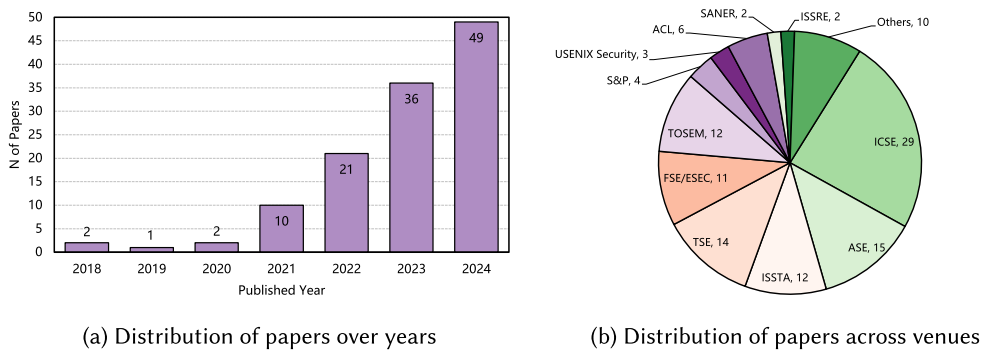


Fig. 1. Overview of papers.

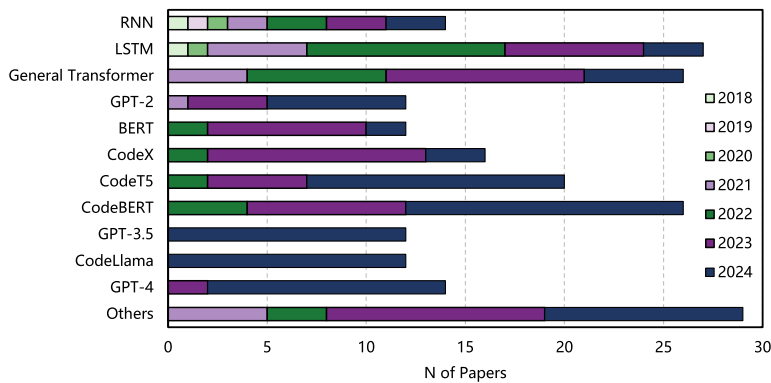


Fig. 2. Distribution of papers across LMs.

Table 1. Distribution of Papers across SE Activities

SE Activities	SE Tasks	Total
Software Development	Code Generation (37)	93
	Code Summarization (20)	
	Code Search (10)	
	Code Completion (8)	
	Code Translation (7)	
Software Quality Assurance	Vulnerability Detection (19)	21
	Clone Detection (15)	
	Program Repair (13)	
Software Maintenance	Defect Prediction (6)	41
	Commit Message Generation (3)	
	Code Classification (5)	
	Code Representation (3)	
	Code Comment Generation (3)	
	Authorship Attribution (1)	
	Named Entity Recognition (1)	
	Test Generation (2)	
	Duplicate Bug Report Detection (1)	
	Bug Report Summarization (1)	
	Bug-Fix Commit Identification (1)	
	Bug Report Classification (1)	

observations from previous surveys in learning-based SE [54, 164, 170]. Table 1 summarizes the distribution of SE tasks addressed in the collected papers. Code generation, code summarization, code search, vulnerability detection, and clone detection emerge as the primary scenarios for investigating LM4Code pitfalls, encompassing both classification and generation tasks.

Overall, our analysis underscores that LM4Code has been an increasing area of interest. Particularly, since 2021, there has been an increasing amount of literature emphasizing the pitfalls of LM4Code. These pitfalls may hinder realistic performance and can affect how reliable and practical

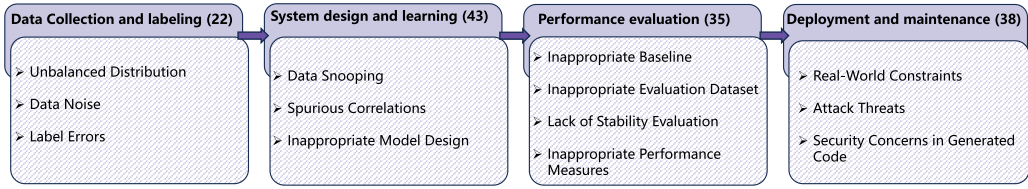


Fig. 3. The overview of pitfalls of LMs for code intelligence.

LM4Code systems are in real-world situations. Our collected papers cover a wide range of LMs and many different SE tasks. As we delve deeper into our primary topic in the subsequent sections, this preliminary overview sets the foundation for a comprehensive analysis of the challenges and opportunities in LM4Code.

2.4 Paper Organization

Through our study collection process, we identified 121 research papers that specifically discuss or address the pitfalls in LM4Code. Similar to prior research [7], our answers are organized following the typical workflow of LMs for code intelligence, which ensures the topics discussed in the collected studies can be covered. To be specific, we segment the pitfalls into four key aspects of the LM pipeline based on the LM4Code lifecycle: data collection and labeling (Section 3), system design and learning (Section 4), performance evaluation (Section 5), and deployment and maintenance (Section 6). This framework is depicted in Figure 3. For each aspect, we first summarize the types of prevalent pitfalls discussed in the collected studies (RQ1) and explore potential solutions and best practices recommended in the literature (RQ2). The implications of these pitfalls (RQ3) are discussed in Section 7, and in Section 8, we further discuss open challenges and promising research directions. This organized structure enables a comprehensive analysis of pitfalls and considerations across the entire LM4Code pipeline. Our taxonomy aims to provide crucial insights for developing more robust, reliable, and practical LM systems for code intelligence tasks.

3 Data Collection and Labeling

The data-hungry LMs require large-scale and high-quality training datasets. According to a survey by Hou et al. [54], the majority of LMs for code intelligence are trained using data from open source platforms, with GitHub and Stack Overflow being the most popular options. However, the data in these platforms are user-contributed, varying significantly in the level of quality and reliability. It leads to non-negligible noises, bias, and errors in the training dataset and further affects the behavior of the models, which brings significant pitfalls in LMs for code intelligence. In this section, we provide a brief description of related studies and discuss the implications and potential solutions during the data collection and labeling stages.

3.1 RQ1—Pitfalls

From the collected papers, we identified 22 research studies focusing on pitfalls during the data collection and labeling process. Table 2 presents the statistics of literature on this topic, where the pitfalls can be grouped into three main categories.

Unbalanced Distribution. Unbalanced distribution arises when there is a lack of proper randomization in the selection of samples, leading to certain populations being underrepresented or overrepresented [134]. In code-related scenarios, it usually refers to the gap between the sample distribution of real-world practices and training datasets. For example, as emphasized by [15, 143, 180], vulnerable instances in vulnerability detection studies are overwhelming while neutral code

Table 2. Summary of Common Biases in Data Collection and Labeling from Reviewed Research Studies

Bias Type	Paper	SE Tasks	Description	Findings
Unbalanced Distribution	[15, 144, 181]	Vulnerability Detection	The ratio of vulnerable and non-vulnerable cases in real-world projects is extremely unbalanced	F1 score drops by 73%
	[188]	Defect Prediction	Many datasets are imbalanced with target classes under 30%	Imbalanced data results in an F1 score below 0.2
	[81]	BRC, Defect Prediction	Class imbalance	/
	[84]	Code Summarization, Clone Detection	Shortage of low-frequency identifiers in model training	/
	[13]	Code Generation	Imbalanced programming languages, Unbalanced code granularity, Lack of advanced features	Pass@5 drops from 90.93% to 48.24%
Data Noise	[96, 192]	Commit Message Generation	Commit messages mix bot-generated content with human-written trivial messages containing redundant or easily inferred information	BLEU-4 drops from 31.92% to 14.19%
	[148]	Code Search	Over one-third of queries of code search datasets contain noises that make them deviate from natural user queries (e.g., HTML tags, interrogation)	MRR improves from 0.407 to 0.512 after data cleaning
	[143]	Code Summarization	Noisy code-comment pairs, including non-literal and duplicated code, are prevalent in four benchmark datasets (31–66%)	Training three models with the cleaned datasets improves the BLEU-4 by 27%, 21%, and 24%
	[48]	Test Generation	Current LLMs are trained on snippets without distinguishing test code from other types	/
Labeling Errors	[81]	BRC, Defect Prediction	Misabeled samples—issue reports that describe defects but are not classified as such	/
	[41, 85]	Code Translation, Clone Detection, Code Search	Most of the collected code snippets are unlabeled	/
	[108, 128]	Vulnerability Detection	Error labels are common in many vulnerability datasets; non-robust features, such as variable names	F1 score drops by 20.7%

instances in real-world environments considerably outnumber their vulnerable counterparts. This imbalance extends to other code-based tasks. In **Software Defect Prediction (SDP)**, where defective modules are scarce compared with non-defective cases in real-world environments [80, 187]. Similarly, **Bug Report Classification (BRC)** suffers from underrepresentation of the minority bug class [80]. In code summarization, the absence of low-frequency identifiers in the training set can lead to a lack of relevant knowledge [83], which may result in hallucinations or be exploited to cause harm. Additionally, the skew in programming languages, the uneven granularity of code, and the absence of sophisticated features within training datasets can also impact large models' comprehension and generation capabilities in relevant tasks [13].

Data Noise. Data noises are the samples that are meaningless or even harmful for the models to learn, such as samples with deprecated coding conventions, multi-lingual comments, and auto-generated code snippets [142]. Such noises widely exist in code datasets. For example, as investigated by Sun et al. [147] and Shi et al. [142], over one-third of the popular code dataset, CodeSearchNet [58], are noises that are hardly seen in neural code search. Their analysis results show that the examined datasets contain a multitude of noise categories, including unrelated comments, non-literal characters, and issues like empty or duplicated code. Liu et al. [95] and Zhang et al. [191] specifically investigated data noise in commit message generation, where approximately 16% of the commit messages of benchmark dataset by Jiang et al. [63] were identified as noise.

Labeling Errors. Labeling errors arise when ground-truth labels are inaccurate, unstable, or erroneous [7, 153]. In some code-related tasks, such as vulnerability detection, the raw code datasets need to be labeled by human annotators. Nie et al. [107] explored the labeling error problem in vulnerability detection wherein a vulnerable code sample is mislabeled as non-vulnerable, and vice versa. They found that mislabeling a non-vulnerable sample as vulnerable was a more pervasive issue. The research further assessed three prominent datasets, D2A [197], Big-Vul [31], and Cross-Vul [108], discovering that in the worst cases, nearly 30% of the labels in these datasets may cause noisy labels. Similarly, Li et al. [80] examined the datasets from Herzig et al. [52] for BRC and from

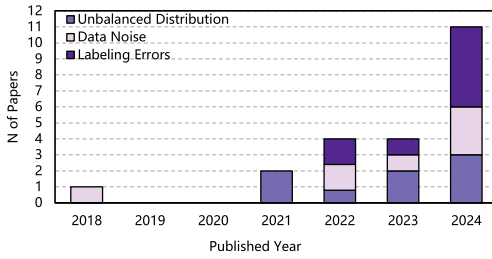


Fig. 4. Paper distribution about data collection and labeling across time (Section 3).

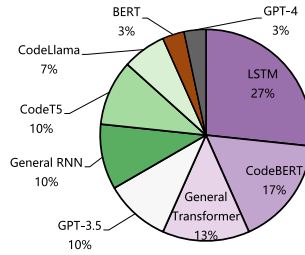


Fig. 5. Distribution of LMs (Section 3).

Yatish et al. [186] for SDP. Their findings revealed that these datasets possess mislabel rates ranging between 2% and 29%.

Overall, Figures 4 and 5 present the distribution of the 22 papers concerning bias in data collection and labeling. Figure 4 shows that the issue of data noise has continuously attracted the attention of researchers and prohibits the sharp surge in research papers related to labeling errors in 2024. Figure 5 as well as Figure 2 reveal that with the emergence and widespread use of sophisticated models such as CodeBERT, CodeLlama, and GPT, the data bias of conventional LMs, specifically LSTM and GRU, is gradually fading from the view of researchers.

3.2 RQ2—Solutions

Recognizing pitfalls in data collection and labeling has emphasized the need for robust solutions to address and mitigate these issues. Several solutions have been proposed in the literature that we've reviewed. These solutions have been organized into distinct categories based on their underlying principles and methodologies.

Data Cleaning/Denoising. Data cleaning/denoising is the process of improving a dataset by removing or correcting abnormalities, inconsistencies, and inaccuracies. This phase is critical to ensure that the training data is accurate and does not contain any misleading or irrelevant information. Many pitfalls develop as a result of noisy or erroneous data, and data cleaning is the major method for dealing with such difficulties. Shi et al. [142] introduced a rule-based cleaning tool, named CAT (Code-comment cleAning Tool), that employs configurable heuristics rules to automatically scan and filter out comments and code with syntactic anomalies, thereby detecting the occurrences and distribution of data noises. Similarly, Sun et al. [147] proposed a two-stage data cleaning framework for code search, employing a rule-based syntactic filter for inconsistent comments followed by a VAE-based semantic filter trained on a bootstrap corpus to prioritize comments with low reconstruction discrepancy. Their evaluation results demonstrate that this hybrid filter approach not only significantly saves computational resources but also enhances model accuracy. Nie et al. [107] introduced confident learning [113] and differential training [177] for denoising-based noisy label detection, aiming to enhance the label quality of vulnerability datasets. They found that the effectiveness of denoising methods heavily relies on the compatibility between vulnerability detection model and its datasets, and these denoising methods show considerable promise in boosting vulnerability detection performance. Furthermore, to accommodate the demands of more granular code analysis tasks and reduce redundancy, contemporary code datasets necessitate slicing at a finer granularity than the function level. This is particularly important for tasks such as binary analysis and vulnerability detection. Consequently, methods for loop-level and line-level code localization and slicing have been proposed [136].

Real-World Benchmarks. To precisely evaluate the models, researchers propose to use real-world benchmarks, instead of train-test-split datasets. Many benchmarks are thus constructed. For example, many benchmarks, such as HumanEval [19], DS-1000 [73], and MBPP [8], are constructed using human-written tasks and test cases to evaluate the code generation LMs. Compared with the code snippets in open source repositories, which can be difficult to classify by task and slice accurately, such human-written code tasks are closer to the real user requests in practice, which better reflects the performance of the model. For instance, Lin et al. [84] use a real-world dataset composed of human-written programming exercises with multiple solutions. Similarly, Mozannar et al. [104, 105] use user behavior data to demonstrate the effectiveness of their proposed methods.

Benchmark Augmentation with LLMs. Despite the many challenges associated with LLMs, they remain highly effective production tools, and we should not dismiss their potential. Leveraging current large models, we can generate more comprehensive, task-adaptive datasets. For instance, Xia et al. [174] introduced EVOEVAL, a program synthesis benchmark suite that uses the power of GPT-4 to evolve existing benchmarks into different target domains, aiming to provide a more thorough evaluation of LLM coding abilities. They found that, compared to the high performance on standard benchmarks like HUMANEVAL [19], popular LLMs experienced a significant performance drop (an average of 39.4%) on EVOEVAL, highlighting the limitations of standard benchmarks and the effectiveness of their approach. Additionally, EvalPlus [89] combines emerging LLM-based methods with traditional mutation-based approaches for test case generation. It continuously expands inputs using LLM and applies a greedy set-covering approach for constraint handling, fully harnessing the capabilities of large models.

Summary - Data Collection and Labeling

Based on 22 relevant studies, our literature review reveals three prevalent pitfalls (i.e., unbalanced distribution, data noise, and labeling errors) in the data collection and labeling process. These pitfalls propagate, causing overestimated performance and compromised model efficacy. Though initial solutions like data cleaning/denoising and real-world benchmarks have been proposed, and recent efforts exploring large models for dataset generation show promise, the field is far from reaching a comprehensive resolution. Furthermore, datasets for SE tasks pose particularly stringent demands on the granularity and accuracy of code snippets, especially concerning the precise identification and slicing of vulnerabilities and malicious code, underscoring the critical need for automated and scalable techniques to ensure high-quality data for LM4Code.

4 System Design and Learning

This section examines pitfalls in the system design and learning process for LM4Code. The training of these LM4Code models directly impacts their quality and efficacy for empowering code intelligence. However, several challenges arise in crafting optimal model architectures, formulating strategic training-testing approaches, refining data preprocessing techniques, and selecting suitable learning algorithms. Each design decision risks introducing pitfalls that can undermine model robustness and effectiveness.

4.1 RQ1—Pitfalls

We have identified 43 research studies dedicated to the exploration of pitfalls introduced in the system design and learning process. These pitfalls can be broadly categorized into three categories: data snooping, spurious correlations, and inappropriate model design. In the following, we provide comprehensive descriptions of these three pitfalls.

Data Snooping. Data snooping arises when LMs are inadvertently exposed during training to information that should be inaccessible. Such unintentional exposure primarily stems from improper data handling. A common source is improper train-test split, where evaluation/testing data leaks into training. For example, research has identified instances where training data incorporates bugs or vulnerabilities that closely mirror those encountered in testing [27, 173]. This concept of overlap is further supported by Liu et al. [87] highlighting the risks associated with functionality similarities between train and test data. Another dimension to consider is the origin of the data. For instance, as emphasized by Steenhoeck et al. [143] and Yang et al. [178], using samples from the same software projects for both training and testing can influence the models. Even data structure and representation choices enable snooping. Liu et al. [94] and Shi et al. [138] emphasize that pitfalls can emerge from the data processing processes such as test prefix generation or concurrent use of same-class data.

Spurious Correlations. Spurious correlations arise when LMs mistakenly depend on irrelevant artifacts rather than the intrinsic logic or intent of code for decision-making, leading to misleading associations. The artifacts vary across SE tasks. For instance, in vulnerability detection, artifacts may manifest as recurring code patterns, semantic redundancy or reliance on specific function names that LMs incorrectly associate with vulnerabilities [15, 135, 143]. In code summarization, models might focus more on strings or certain code structures while overlooking elements key for developers [118]. When generating commit messages in the context of code review, models often produce outputs that adhere to a few simple patterns, potentially failing to capture the nuances of the actual code changes [28]. Actually, introducing advanced pre-trained models like CodeBERT has not eliminated these pitfalls. Specifically, if not fine-tuned appropriately for downstream tasks, these models might still overemphasize basic elements like keywords over richer code semantics [196].

Inappropriate Model Design. Inappropriate model design in LM4Code arises when the underlying architecture fails to capture critical characteristics of code, such as hierarchy and composition. The inability to construct robust semantic representations of code's intricate structural and logical attributes hinders model efficacy on downstream code intelligence tasks. Such design shortcomings can manifest in several ways. For instance, in vulnerability detection, models may exhibit a significant overlap in the feature space between classes, hindering precise vulnerability identification [15]. Code search models might lean on coarse-grained representations, capturing merely lexical or structural elements, often overlooking the true functionality of the code [162]. Similarly, the encoder-decoder framework used in code summarization might neglect the hierarchical nature of code or struggle with sequence generation, leading to inadequate summaries [160, 168]. This issue is not limited to traditional architectures. Even modern program repair models, adapted from neural machine translation, face design-related challenges that affect their translation accuracy and diversity [27, 102]. While there are innovative attempts such as leveraging deep reinforcement learning or shared encoder-decoder architectures [84], these approaches still exhibit shortcomings in addressing the diverse needs of various LM4Code applications.

To summarize, Figures 6 and 7 display the distribution of papers that discuss LM4Code pitfalls in the system design and learning stage. Figure 6 indicates that while inappropriate model design was first identified in 2018, research efforts on addressing key pitfalls have increased over the past 3 years. Among these, data snooping has garnered increasing research attention. Meanwhile, spurious correlations have become more prominent with the advent of **Explainable AI (XAI)** techniques for elucidating model reasoning [22, 150, 152]. Discussions around inappropriate model design remain ongoing as new frameworks and learning strategies continue to emerge. Similar to our observations regarding the data collection and labeling process, Figure 7 reveals a greater emphasis on modern Transformer-based LMs and GPT series compared to conventional architectures. This distribution highlights a shift towards examining potential pitfalls in sophisticated LMs

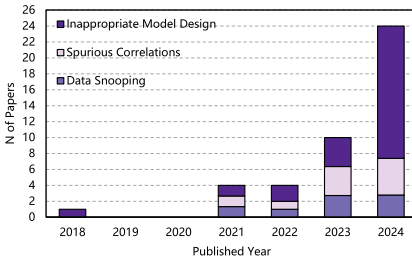


Fig. 6. Paper distribution about system design and learning across time (Section 4).

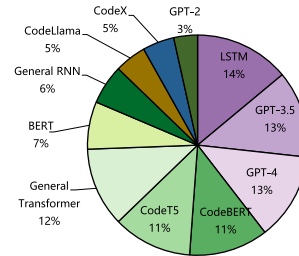


Fig. 7. Distribution of LMs (Section 4).

for code intelligence tasks, setting the stage for continued research focused on enhancing model transparency, interpretability, and reliability.

4.2 RQ2—Solutions

To address the three pitfalls related to the system design and learning process, researchers have employed a variety of approaches which we describe as follows.

Refined Data Handling. To mitigate the challenges posed by data snooping, a multifaceted approach is essential. Emphasizing rigorous data partitioning is foundational, as exemplified by Steenhoek et al. [143], who advocate for cross-project validation to prevent inadvertent overlaps between training and testing sets. Additionally, techniques like data augmentation and time-based splits can further insulate models from over-relying on specific pattern [45, 121]. Incorporating regularization techniques, such as batch normalization, can curb overfitting and deter models from exploiting inadvertent data correlations [49]. Finally, the importance of external validation, underscored by Liu et al. [87], ensures that performance assessments are unbiased and reflective of real-world scenarios. By adopting these strategies, researchers can foster more reliable and generalizable outcomes.

Model Interpretability. To address the biases inherent in LM4Code, especially spurious correlations, improving model interpretability has emerged as a crucial solution [150]. By examining the decision-making process of LM4Code models, researchers are better positioned to pinpoint and mitigate pitfalls, leading to more reliable predictions [50]. Within vulnerability detection, Fu et al. [38], Li et al. [79], and Zou et al. [201] proposed methods to enhance explanation accuracy, leveraging sophisticated visualization tools to correlate the internal dynamics of neural models with code structures, thus providing a comprehensive understanding of model reasoning. Cito et al. [23] offer a distinctive perspective, centering on elucidating mispredictions. Their approach, which integrates neural predictions with symbolic logic, allows for precise error detection accompanied by rule-based explanations. Additionally, attention mechanisms to explain pre-trained models have also been analyzed. While Shi et al. [139] unravel how transformer-based models allocate attention for code summarization, Wan et al. [159] probe into the nuances of attention during code-to-code translation. These XAI approaches can serve to identify and rectify model pitfalls, ensuring the reliability of LM4Code applications.

Model Optimization Strategies. In light of the pitfalls introduced by inappropriate model design, researchers have turned to model optimization strategies to address and minimize their effects. These strategies encompass several techniques designed to enhance a model's structure, training process, and generalization capabilities. Firstly, model design adjustments involve refining the architecture to better capture data intricacies. Studies like that by Wan et al. [160] have demonstrated

the benefits of introducing novel layers or structures to better understand the tree structure of code, yielding improved performance. Secondly, model ensembling is gaining traction, where the strengths of multiple models are leveraged to offset individual biases, as seen in the work by Zhang et al. [196] which employs multiple views of the same data for more robust predictions. Lastly, regularization and fine-tuning techniques play a pivotal role. Regularization, such as dropout or L2 regularization, helps in preventing overfitting, while fine-tuning allows pre-trained models, like CodeBERT, to adapt to specific dataset nuances, as demonstrated by Fang et al. [33]. By integrating these strategies, models can be better positioned to achieve superior outcomes.

Summary - System Design and Learning

In this study, we uncover 43 research studies related to pitfalls in system design and learning. These pitfalls can be categorized into three main categories: data snooping, spurious correlations, and inappropriate model design, leading to overestimated performance and compromised efficacy of LM4Code Systems. Proposed solutions encompass refined data handling, model explainability, and optimization strategies like architecture adjustments, ensembling, and regularization. For SE tasks specifically, selecting an appropriate model is crucial, necessitating consideration of domain knowledge specific to particular code types, the complexity of the coding task, and model input limitation.

5 Performance Evaluation

The performance evaluation stage focuses on precisely assessing and analyzing the model's performance using predefined test sets and evaluation metrics. Additionally, comparative performance evaluation against benchmarks provides insights into a model's strengths and weaknesses on specific code-related tasks. However, potential pitfalls can emerge from factors such as improper baselines, test sets, and performance metrics. These challenges must be thoroughly examined and addressed to ensure that the evaluation is unbiased, comprehensive, and representative of a model's true capabilities. Thus, this section provides a brief description of related studies and discusses the implications and potential solutions during the performance evaluation stages.

5.1 RQ1—Pitfalls

From the collected studies, we identified 35 research studies focusing on pitfalls during the performance evaluation phase. We have methodically categorized the collected literature into four categories, as shown in Figure 8.

Inappropriate Baseline. Inappropriate baselines arise when the performance evaluation for LM4Code is carried out without, with limited, or with skewed baseline approaches. Such poor comparisons fail to convincingly demonstrate the improvements or strengths of newly proposed LM4Code approaches, potentially leading to misguided experimental findings or exaggerated efficacy claims. For instance, comparing only against basic rule-based approaches would inflate capabilities, while limiting comparisons to other advanced LM-based approaches masks potential weaknesses. Liu et al. [94] highlight this issue in state-of-the-art Neural Test Oracle Generation evaluation, where the lack of a straightforward baseline causes considerable gaps between reported and real-world performance.

Inappropriate Evaluation Datasets. Inappropriate evaluation datasets refer to the use of unsuitable, non-representative, or limited test sets that fail to adequately represent the true complexities of the studied tasks. Such test sets can provide misleading results and offer an obscured, over-optimistic, or pessimistic view of a model's realistic capabilities. Liu et al. [88] highlight the frequent use of small,

non-representative, and non-diverse datasets in code generation evaluations that fail to capture real-world software complexities. Specifically, the widely used HS dataset [85] comprises code from a single project, exhibiting poor diversity. The average code length in the Django dataset [114] is a mere 33 characters, indicating limited program complexity. The CoNaLa dataset [188] utilizes automatically extracted Stack Overflow questions instead of real software requirements. In short, common benchmarks poorly approximate the complexities of realistic code generation scenarios. In vulnerability detection, Liu et al. [90] highlight that most evaluations assume that training and test datasets are drawn from the same distribution. However, they argue that this assumption overlooks the continuous evolution of software vulnerabilities and projects, leading to the Cross-Domain issue, where test sets should contain novel vulnerabilities or projects. Furthermore, Nong et al. [112] further reveal that vulnerability datasets like SARD [12] comprise unrealistic synthetic examples, which exhibit smaller vocabulary, smaller program length, and higher pattern frequency compared to real-world code. Zeng et al. [190] claim that the state-of-the-art Just-in-Time defect prediction tool, CC2Vec [53], was only evaluated on a limited dataset with marginal improvements to demonstrate generalizability and scalability. Overall, common benchmarks in code-based research utilize inappropriate test sets that fail to capture real-world complexities, leading to unrealistic performance evaluation.

Lack of Stability Evaluation. Despite the widespread adoption and praise for large models, their inherent randomness has been heavily criticized for causing output instability, with a lack of evaluation and proof of model stability. This randomness can lead to inconsistent results even in repeated experiments of the same study, raising concerns about the validity and generalizability of scientific research based on large models. Additionally, large model-based research faces specific challenges in effectively storing big data, managing communication between distributed clusters, implementing algorithms, and ensuring appropriate software/hardware environments [64, 74]. As a result, the randomness in LM4Code models significantly affects performance evaluation, further impacting the generalizability and trustworthiness of research findings. Zeng et al. [189] conducted a comprehensive empirical study, revealing reproducibility issues with pre-trained large models in tasks like program understanding and generation. Their study showed that multiple performance comparison results contradicted those in original publications. For instance, across five clone detection runs on the BigCloneBench benchmark, CodeGPT exhibited the largest variance, with performance either surpassing or underperforming other models depending on the run. Although the original paper reported that PLBART had a 0.7 F1 advantage over CodeBERT, the min-max F1 variance of 0.84 brings this claim into question. Furthermore, the defect detection accuracy of CodeBERT and PLBART was reversed between the original PLBART paper and the subsequent research by Zeng et al. [189]. Such significant statistical fluctuations can overturn performance comparisons, ultimately undermining the credibility of LM4Code.

Inappropriate Performance Measures. Inappropriate performance measures arise from a mismatch between standardized metrics and the distinct challenges inherent to SE tasks. Prior survey studies [54, 166] have shown widespread use of generic metrics like accuracy, precision, BLEU, and Pass@k in LM4Code research. However, these metrics often provide an incomplete view of model capabilities on code-based tasks, owing to the complex and multifaceted nature of SE scenarios. For instance, Roy et al. [130] conducted an empirical study with 226 human annotators and showed that popular metrics like BLEU and ROUGE are poor reliable indicators of human judgment. This demonstrates how common metrics may not fully align with code QAs. Additionally, when evaluating classification tasks like vulnerability detection, reporting solely accuracy is insufficient, as true-positive and false-positive decisions are not observable [15, 94]. Moreover, popular metrics like BLEU derived from natural language processing overlook critical attributes of code. As a textual similarity metric, BLEU calculates n-gram precision between generated and reference

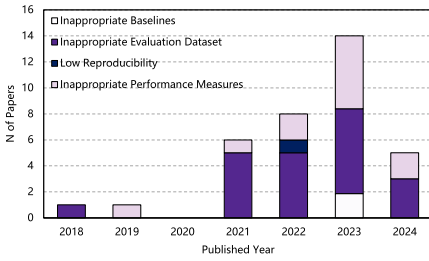


Fig. 8. Paper distribution about performance evaluation across time (Section 5).

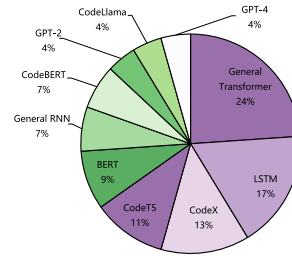


Fig. 9. Distribution of LMs (Section 5).

sentences. However, programming languages contain many “trivially shared n-grams,” rendering BLEU ineffective at distinguishing actually similar code from coincidental similarities [30]. As we discussed before, code-based tasks exhibit multiple distinct challenges such as data imbalance, data snooping, execution correctness, and exception handling. For example, despite high BLEU scores, CodeT5 generated code with only 6.4% compilation rate, indicating poor execution correctness [173]. Overall, it is critical to utilize appropriate evaluation metrics in LM4Code research to provide a comprehensive and accurate understanding of model performance.

Figure 8 presents the distribution of the relevant literature over years, while Figure 9 summarizes the distribution between different LMs. Between 2018 and 2023, there was a noticeable increase in the number of articles, suggesting a growing focus of researchers on pitfalls in LM4Code during the performance evaluation phase. However, in 2024, there was a sharp decline, with an Inappropriate evaluation dataset and Inappropriate performance measures contributing significantly to this decrease. Among the identified studies, inappropriate evaluation datasets have the most mentions, with 20 studies, showing the difficulty in establishing representative datasets of LM4Code for reliable model performance. Additionally, inappropriate performance measures also garnered considerable research attention, with 16 identified studies. From Figure 9, we can observe that Transformer-based models were the most discussed, evidencing their increasing adoption for SE tasks. In summary, these observations emphasize critical areas for improvement to enable robust evaluation and unbiased analysis of capabilities as LM4Code evolves. Careful consideration of evaluation datasets, metrics, and baselines will be integral to advancing progress in the field.

5.2 RQ2—Solutions

Given the pitfalls identified during the performance evaluation phase, it is essential to propose solutions to address these challenges and optimize the evaluation process. We summarize the existing solutions as follows.

Standardized and Realistic Benchmarks. Establishing robust and realistic benchmarks is critical for rigorous performance evaluation in LM4Code research. Researchers should carefully select appropriate baselines based on thorough literature analysis, opting for well-established approaches tailored to specific code intelligence tasks. This enables comprehensive capability assessment. Moreover, the community must collaboratively institute standardized benchmarks and protocols, fostering consistent evaluation. For example, Liu et al. [94] highlight the need for establishing straightforward and realistic baselines to ensure truthful evaluation outcomes. For specific tasks, standardized task-specific benchmarks have emerged, such as VJBench proposed by Wu et al. [173] for automated program repair. Such domain-specific benchmarks facilitate targeted capability assessment. Furthermore, diverse benchmark datasets play a significant role in comprehensively

evaluating and advancing LM4Code. For instance, Lu et al. [98] introduced CodeXGLUE, a comprehensive collection that encompasses 10 SE tasks across 14 datasets. It provides a platform for structured model evaluation and comparison across tasks using standardized baselines.

Enhancing Reproducibility. Inconsistent replication remains a key pitfall during performance evaluation in LM4Code [64]. Some solutions to enhance reproducibility have been proposed. At its core, it's critical that code, datasets, and evaluation scripts are made publicly accessible, echoing the call for open source initiative and providing continuously maintained links to high-quality replication packages [86]. Furthermore, reporting key statistical measures like variance, confidence intervals, and significance quantifies stability across runs. Such transparency fosters community collaboration, facilitating the validation of research outcomes and paving the way for future scholarly advancements. Carefully controlling and reporting experimental conditions like hardware specifications, software versions, random seeds, and tuning details significantly influences outcomes [59]. Chen et al. [18] introduced a systematic approach to train reproducible AI systems, with general criteria to evaluate reproducibility, and a unified framework to decrease randomness. Finally, stability evaluations such as variance [189], standard deviation, confidence intervals, and top-K consistency can be used to evaluate the stability of LMs' output, thereby improving the reproducibility of LM4Code.

Refined Performance Measures. Appropriate and comprehensive performance metrics are critical for accurately evaluating model performance on complex code-related tasks. To address the limitations of common generic metrics like BLEU and accuracy, researchers have proposed more refined measures tailored to LM4Code challenges. Ren et al. [128] introduced CodeBLEU to improve upon BLEU for assessing deeper semantic similarities in code generation tasks, while Eghbali et al. [30] introduced CrystalBLEU for minimizing the noise from commonly shared n-grams in programming languages. Given the runtime nature of programming languages, metrics like compilation rate and execution accuracy on test cases have also been used [173]. For specific LM4Code tasks, domain-specific metrics can prove more reliable. For example, for test oracle generation, Liu et al. [94] presented an evaluation metric named Found@K, which counts how many bugs can be found if developers only check the top-K recommended test cases per bug. To better capture complex code quality attributes, direct human assessment can complement automated metrics, evaluating aspects like readability, conciseness, and correctness [26, 118]. Overall, more refined, code-aware, and multifaceted performance measures aligned with practical goals provide a more accurate and reliable model capability assessment.

Summary - Performance Evaluation

In the performance evaluation of LM4Code, we identify four pitfalls related to performance evaluation, including inappropriate baseline, inappropriate evaluation datasets, low reproducibility, and inappropriate performance measures. From 35 relevant research studies, inappropriate datasets and metrics receive much attention. Such pitfalls can lead to overestimated evaluation and compromised reproducibility, misleading benchmarks for the future. To address this, tailored solutions like standardized benchmarks, transparency, realistic assessments, and community coordination are needed, particularly given the unique challenges of evaluating LLMs for code.

6 Deployment and Maintenance

Systems based on advanced LM4Code, such as GitHub Copilot [43], Amazon CodeWhisperer [5], and Microsoft IntelliCode [148], have already been deployed in real-world IDEs and have garnered a large number of users. There exist various challenges when such LM4Code systems are deployed in

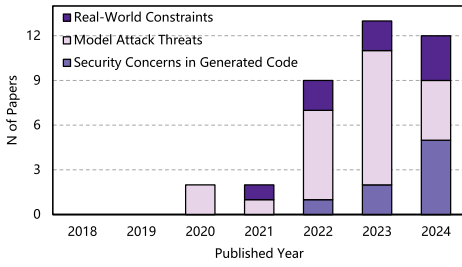


Fig. 10. Paper distribution about deployment and maintenance across time (Section 6).

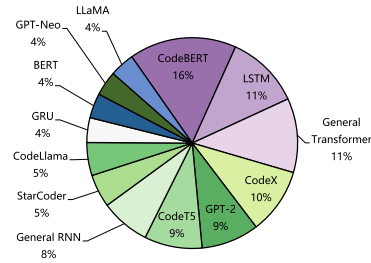


Fig. 11. Distribution of LMs (Section 6).

practice, like security threats, and how LM4Code systems should be updated to adapt to the rapidly changing software practices. This section discusses pitfalls, implications, and current solutions when deploying and maintaining LM4Code.

6.1 RQ1—Pitfalls

In our literature review, we identified 38 research papers involving the pitfalls of LM4Code deployment and maintenance. This number of identified publications surpasses that related to other stages, highlighting the fact that deploying and maintaining LM4Code presents a complex set of challenges that have gained considerable attention from the research community.

Figures 10 and 11 present the distribution of these 38 research studies over time and different types of LM4Code. From Figure 10, we can observe that there has been a significant increase in the number of research papers focusing on the pitfalls in the deployment and maintenance of LM4Code, especially in the past 3 years. The data presented in Figure 11 demonstrates that large pre-trained models such as CodeBERT, General Transformers, and Codex appear to surpass traditional LMs such as RNN and LSTM in terms of widespread use and popularity. These observed patterns indicate a shift in research focus towards investigating the complexities of advanced models when deployed in real-world contexts. In the subsequent sections, we will examine the particular biases and issues that arise in regard to the development and maintenance of LM4Code.

Real-World Constraints. Evaluating LM4Code systems solely in controlled lab settings often overlooks practical constraints and complexities of real-world deployment [7]. Although controlled evaluations provide useful insights into the effectiveness of a model within specific settings, they sometimes overlook the complexity and diversity of real-world deployments. It is important to consider runtime and storage constraints when deploying an LM4Code system in the wild. Svyatkovskiy et al. [149] mentioned that a reasonable upper bound of model size for an IDE plugin is 50 MB. However, the size of LM4Code keeps increasing. For example, the popular LM4Code, CodeBERT [36], has 125 million parameters and is 425 MB in size, which is way larger than the suggestion by Svyatkovskiy et al. [149]. Recently proposed models are even larger. Considering LLaMa, developed by Meta, its smallest version has 6.7B parameters [97]. Models that appear to have good performance in a controlled setting may be resource-intensive or excessively large for practical applications, especially in environments with limited computational power or storage capacity [35].

Attack Threats. LM4Code systems face various threats from malicious attackers. Despite the breakthrough capabilities exhibited by LMs, prior works have noted that the state-of-the-art LMs are vulnerable to a variety of attack threats such as evasion attacks [101, 163, 199], backdoor attacks, privacy attacks, etc. These threats can appear across the entire lifecycle of the model, from data collection to model deployment. We identified a total of 17 research papers that specifically focus

on the analysis of security threats within the LM4Code system. Figure 10 shows that the number of these studies has increased over the recent years. Our study employs the classification scheme mentioned in the work of Battita et al. [11], which has identified a diverse range of attack types in machine learning.

- (1) *Evasion Attacks.* Evasion attacks (a.k.a. adversarial attacks) leverage adversarial samples [91, 199], which are carefully perturbed instances while appearing as regular and benign inputs to the human observer. Yet, these small perturbations can mislead the trained LM4Code model to produce incorrect predictions. As shown in Table 3, evasion attacks are a major concern, with 9 out of the 21 examined papers investigating them. Zeng et al. [189] thoroughly evaluated eight different adversarial attack approaches (i.e., word importance rank, genetic algorithm, random substitution) against widely used LM4Code, including CodeBERT, GraphCodeBERT, and CodeT5. Their findings emphasized the vulnerability of these models to semantics-preserving adversarial samples. Interestingly, even simple random attack techniques showed significant effectiveness in degrading pre-trained LM4Code.
- (2) *Poisoning Attacks.* Poisoning attacks inject malicious examples into the training data, aiming to manipulate the model's behavior [11, 146]. This can lead the model to produce incorrect or attacker-chosen outputs when certain triggers appear in the inputs [132]. Poisoning attacks can be categorized into two main classes: untargeted poisoning attack and targeted poisoning attack [155]. One special case of the target poisoning attack is the backdoor attack, where adversaries carefully insert a distinct pattern into a subset of training samples to embed a backdoor. When the pattern appears, the model produces pre-defined outputs (e.g., recommending vulnerable APIs). Otherwise, the model behaves normally. Table 3 lists eight papers related to poisoning attacks, with all of them specifically focusing on backdoor attacks. Among various code scenarios, three studies [144, 146, 158] target LMs-based code search models, demonstrating that backdoor samples within code search tasks closely resemble clean code and are not easily differentiated. Four studies [78, 122, 132, 146] focus on code generation scenarios, demonstrating that attackers can manipulate code recommendations.
- (3) *Privacy Attacks.* Privacy attacks refer to attacks aiming to infer the private information of LM4Code, e.g., parameters of models that are hosted remotely and the model training data. One such example is model stealing (a.k.a. model extraction) involves extracting knowledge (e.g., hyperparameters, model architecture, and training data) from a trained model without direct access to its parameters or training data [14, 116]. Adversaries use this technique to “copy” the functionality of a model, often by querying it repeatedly and analyzing the outputs [116]. For example, Lukas et al. [99] delved into the risk of LMs, like GPT-2, leaking personally identifiable information. Yang et al. [184] found that simply extracting 20,000 outputs (each having 512 tokens) from CodeParrot [1] can produce over 40,125 code snippets that are memorized from its training data. In addition, Niu et al. [111] introduced and evaluated a semi-automated pipeline that employs a membership inference approach on various code generation models like CodeParrot [1] and Polycoder [176]. By leveraging the GitHub Search API's hit rate as a distinguishing heuristic and incorporating human-in-the-loop evaluations, they found that approximately 8% (43) of the prompts in the Codex model, used in GitHub Copilot, resulted in privacy leaks.

Security Concerns in Generated Code. The outputs from LM4Code, i.e., generated code, will be further used in other software systems. Consequently, the safety and robustness of the generated code come under scrutiny. Several recent investigations have shown that LM-generated code can contain vulnerabilities, emphasizing the need for rigorous validation and enhancement of their outputs. Pearce et al. [119] analyzed the generated code of GitHub Copilot and identified security

Table 3. Summary of Attack Threats in Reviewed LM4Code Papers

	Category	Percentage	References and Details
Attack Objectives	Evasion Attacks	43%	[40, 51, 92, 168, 183, 190, 194, 195, 200]
	Poisoning Attacks	38%	[76, 79, 123, 133, 145, 147, 159, 184]
	Privacy/Confidential Attacks	19%	[4, 57, 100, 112]
Attack Models	RNN-Based (e.g., LSTM)	31%	[40, 51, 76, 133, 159, 194, 195, 200]
	General Transformer	23%	[40, 57, 159, 168, 190, 200]
	LLM (e.g., Codex and Llama)	46%	[4, 57, 76, 79, 100, 112, 123, 133, 145, 147, 184, 190]
Countermeasure-Inclusive	Include	67%	[40, 51, 57, 76, 92, 100, 133, 145, 147, 159, 183, 194, 195, 200]
	Non-Include	33%	[4, 79, 112, 123, 168, 184, 190]

vulnerabilities. The authors produced 89 different scenarios for Copilot to complete, resulting in 1,689 programs. Alarmingly, they found approximately 40% of these to be vulnerable. Such vulnerabilities arise because code often contains bugs, and given the vast quantity of unvetted code that Copilot processes, the LM is prone to learning from exploitable and buggy code.

6.2 RQ2—Solutions

6.2.1 Improving Model Robustness. Improving model robustness is essential to defend against evolving security threats targeting LM4Code systems. Here, we summarize some of the available solutions that have been proposed to enhance the robustness of LM4Code models:

Adversarial Training. Adversarial training incorporates adversarial examples into training data to improve model robustness against adversarial attacks. Regarding code comment generation, Zhou et al. [199] demonstrated masked adversarial training can significantly enhance robustness while retaining performance on normal test data. Zhang et al. [194] illustrated that adversarially trained LMs exhibit markedly reduced attack success rates (approximately 30–40%). Moreover, Henkel et al. [51] implemented robust optimization employing a semantics-preserving adversary, and this approach outperformed standard data augmentation, optimally balancing accuracy on clean samples with robustness to perturbations.

Inference with Self-Repair. Self-repair is to let the model introspect and correct mistakes or vulnerabilities in its own code. Delving into this, Olausson et al. [115] investigated to what extent the GPT models can provide accurate feedback on the causes of code errors, validating the efficacy of GPT-4’s self-repair capabilities. Moreover, Chen et al. [17] enhanced the self-repair process by incorporating knowledge from human-written natural language feedback.

Domain Knowledge. Beyond the internal capability of the model, external aids with domain knowledge can also be employed to address the challenges. For instance, Jain et al. [61] enhance these LLMs by utilizing a post-processing step based on program analysis and synthesis techniques, thereby improving the correctness of the syntax and semantics in code. Wei et al. [171] introduced Repilot, a framework designed to further assist AI “copilots” (i.e., LLMs) by synthesizing more valid patches during the repair process. Moreover, Johnson et al. [66] proposed Random Utility-Driven Synthesis Under Uncertain Regions (R-U-SURE), a method that builds uncertainty-aware suggestions based on a decision theory model of objective conditional utility, using random samples from generative models as proxies for unobserved potential intents of end-users. Poesia et al. [123] propose a framework to apply constraints on partial outputs to produce complete correct programs without re-training or fine-tuning of the LM.

Train with Real-World Dataset. Directly learning from the real-world datasets helps the model to better align with the user intention. For instance, Aye et al. [9] observed a significant decline in the accuracy of Transformer sequence models when tested using real-world data from production logs. Moreover, they demonstrated that training on real-world examples yields a more robust

model. However, there is a significant shortage of large-scale, real-world datasets, especially for vulnerability. Nong et al. [112] suggest a promising alternative solution: using deep learning to generate real-world samples.

6.2.2 Enhancing Computational Efficiency. To make full use of the limited resources, various measures have been proposed.

Model Compression and Specialization. To enable the deployment of LLMs in resource-constrained environments, model compression techniques have been widely explored [47, 124]. Shi et al. [141] introduced Compressor, which uses a genetic algorithm to guide the simplification of pre-trained code models. This compressed models to significantly smaller sizes with acceptable accuracy loss. In addition to compression, methods to specialize LLMs for specific tasks show promise. **Parameter-Efficient Fine-Tuning (PEFT)** effectively adapts LLMs using limited task data. As Weyssow et al. [172] demonstrated, PEFT outperforms approaches like Incremental Curriculum Learning in reducing computational overhead and boosting performance when specializing broad LLMs.

Efficient Inference. Numerous inference optimization strategies have also been proposed. For instance, Zhang et al. [196] adopted input simplification strategies like word dropout and frequency filtering to simplify input programs. By focusing on the most informative tokens, computational costs are significantly reduced. Additionally, Chirkova et al. [21] introduced a tokenization method that reduces the average token length by 17% without any downstream performance loss. They further demonstrated that a properly chosen tokenization can even enhance the model's performance by 0.5–2%. Furthermore, Svyatkovskiy et al. [149] unveiled an innovative neural completion model by combining static analysis with granular token encoding. This model boasts a lean memory footprint, consuming just 6 MB of RAM—a significant 19x reduction compared to previous models. It can generate a single piece of code completion in a mere 8 ms and delivers an impressive 90% accuracy rate for its top five suggestions.

6.2.3 Privacy and Copyright Protection. The ability of LLMs to memorize and reproduce training data raises critical privacy and copyright concerns. Thus, developing effective protection for privacy and copyright has been widely investigated.

Privacy-Preserving Techniques. The memorization and regeneration capabilities of LLMs raise critical privacy concerns that demand research attention. Some state-of-the-art models like StarCoder employed human annotators to mask any personal information such as keys and addresses present in the training data, in an effort to mitigate privacy risks [76]. In addition, differential privacy techniques have emerged as promising strategies for mitigating privacy risks. Lukas et al. [99] introduce a sentence-level differential privacy approach, which provides guarantees under the assumption that records are unlikely to be duplicated. Their results show that while helpful in reducing privacy leakage to a large extent, differential privacy alone cannot completely eliminate risks. Further research into complementary privacy-preserving mechanisms is needed to develop LM4Code that generates high-quality outputs while provably protecting user privacy.

Model Obfuscation. Obfuscating models by hiding their structure and parameters has been proposed as a technique to protect against extraction attacks. Prior research has shown that attackers can easily craft white-box-like attacks against models on devices, even to the extent of reversing their training data [56]. For example, Zhou et al. [198] developed ModelObfuscator to apply techniques like model file obfuscation and model structure obfuscation. In model file obfuscation, they utilized renaming, parameter encapsulation, and neural structure obfuscation approaches, effectively obfuscating the data and structures of on-device models. Model structure obfuscation utilizes shortcut and extra layer injection, making reverse engineering harder. Although model obfuscation has shown promise by Zhou et al. [198], it can increase library size, introducing

extra memory overhead and computation time. Further research is needed to balance security, efficiency, and accuracy as model obfuscation techniques are applied.

Watermarking. Watermarking techniques can help protect model intellectual property. For instance, Sun et al. [146] introduced CoProtector, which uses data poisoning to embed watermarks into source code repositories. This ensures open source code can't be exploited by models while providing a way to reveal watermark backdoors. However, CoProtector notably diminishes model performance, making it hard to adopt broadly. In neural code completion, Sun et al. [145] proposed CodeMark which embeds imperceptible user-defined watermarks into code. This traces code utilization while meeting key watermark properties like harmlessness, verifiability, and robustness.

Summary - Deployment and Maintenance

Based on 38 relevant studies, our literature review reveals three main pitfalls in deploying and maintaining LM4Code systems: real-world constraints, attack threats, and generated code containing security concerns. These pitfalls can lead to untrustworthy results, copyright infringement, and reduced model efficacy. Proposed solutions involve improving robustness through adversarial training and error detection, enhancing efficiency via compression and optimized inference, and protecting privacy and copyright through obfuscation and watermarking. However, there remains a need for robust evaluation frameworks and techniques that balance security, efficiency, and accuracy. The implications highlight that real-world deployment introduces complex challenges for LM4Code systems.

7 RQ3—Implications

The implications of pitfalls in LM4Code span across multiple dimensions, from direct technical impacts to broader research validity concerns. The ramifications of these pitfalls and biases extend beyond merely impacting research performance and reproducibility from an evaluative standpoint. They also contribute to misleading benchmarks and potential security vulnerabilities within the generated code, when viewed from the perspective of content generation. Furthermore, concerning research performance, a distinction can be made based on whether the evaluation or the model itself is affected, leading to categories such as performance overestimation and compromised model efficacy. These implications collectively demonstrate how pitfalls encountered at various stages of the lifecycle can undermine the reliability and practical applicability of LM4Code systems.

7.1 Performance Overestimation

Performance overestimation occurs when reported results present an artificially optimistic view of model capabilities due to methodological oversights and experimental biases. This phenomenon represents a significant challenge in LM4Code research, where the gap between published results and real-world performance can mislead both researchers and practitioners about the actual capabilities of these systems. Our analysis reveals performance overestimation across multiple phases of LM4Code development.

Data Collection Phase. Train-test-split practices, while common in neural model evaluation [7], can lead to significant overestimation when test sets inherit the same biases as training data. For instance, vulnerability detection models evaluated on balanced datasets achieve impressive F1 scores exceeding 90% [15]. However, when tested on realistic datasets containing only 6% vulnerable examples, their F1 scores plummet to 18.47%, despite maintaining high recall rates of 91.24%. Similarly, Zhang et al. [191] demonstrated how Transformer-based models trained on noisy datasets (including bot-generated and trivial messages) achieved a BLEU-4 score of 42.4%,

which dropped dramatically to 26.2% after noise removal. These cases illustrate how data sampling imbalances and noise can lead to exaggerated performance metrics in LMs for code tasks.

System Design Phase. Pitfalls in system design and learning can lead to over-optimistic performance metrics for LM4Code models. Data snooping is a major contributor to this overestimation. For example, the presence of overlapping functionality between training and test sets can elevate the F1 score of a clone detection model from 0.42 to 0.96 [87]. In vulnerability detection, a mix of projects in both training and testing phases can introduce discrepancies in F1 scores as large as 0.32 [143]. Spurious correlations represent a subtler and often more elusive challenge. These pitfalls cause models to make correct predictions, but often for the wrong reasons, leading them to rely on irrelevant code patterns or unrelated artifacts. This not only misleads performance interpretation but also makes the models unreliable in varied scenarios [143]. Models might also give undue attention to superficial code constructs, leading to inefficiencies that don't necessarily enhance outcomes [196].

Performance Evaluation Phase. Inappropriate and unrealistic datasets can cause an overestimation of model capabilities, leading to a misleading gap between reported metrics and real-world performance. For example, Liu et al. [88] demonstrated that when code generation models were tested on small, synthetic datasets like Django [114] and HS [85], they achieved BLEU scores of 0.811 and 0.646, respectively. However, switching to a more practical dataset caused the BLEU score to drop dramatically to 0.167, revealing the inflated nature of initial results. Similarly, Ahmed et al. [3] found that in program repair tasks, as input token lengths increased, model accuracy dropped from 82.88% to 55%, showing how evaluation metrics fail to account for real-world complexities. Popular metrics like BLEU also fall short, as Eghbali et al. [30] showed that a dummy model exploiting common n-grams could achieve similar scores to a neural model, despite producing low-quality code. This highlights the need for more rigorous and realistic benchmarking to avoid exaggerated assessments of LM4Code models.

These findings underscore the urgent need for more rigorous evaluation protocols and realistic benchmarking standards in LM4Code research. The cumulative effect of performance overestimation across different phases creates a significant disparity between reported capabilities and practical effectiveness, potentially misdirecting research efforts and deployment decisions.

7.2 Compromised Model Efficacy

Model efficacy in LM4Code systems can be significantly compromised by limitations and constraints across different phases of development, leading to reduced performance and reliability in real-world applications. Unlike performance overestimation issues that arise from evaluation biases, these limitations represent genuine constraints that directly impact model capabilities and practical effectiveness. Our analysis reveals how model efficacy is compromised across three key aspects of LM4Code development.

Data Collection Phase. High-quality training data serves as the foundation for trustworthy model training, yet inherent noise and errors can severely compromise model efficacy by causing models to learn irrelevant patterns or establish spurious correlations. Sun et al. [147] demonstrated that when code search models are trained on carefully cleaned datasets, their performance improves significantly, with MRR rising from 0.407 to 0.512. Similarly, Nie et al. [107] revealed that labeling errors can drastically reduce the performance of vulnerability detection models, leading to an average F1 score drop of 20.7%. These examples underscore the importance of high-quality data for ensuring reliable model performance.

System Design Phase. Inappropriate model design directly compromises the efficacy of models in practical scenarios. For example, token sequence-based vulnerability detection models might fail to capture the underlying causes of vulnerabilities and instead focus on surface-level patterns,

leaving a significant margin for improvement. Chakraborty et al. [15] highlighted this limitation and showed that the use of gated graph neural networks can improve the baselines by 33.57% in precision and 128.38% in recall. Similarly, the design limitations in code search models lead them to generate coarse-grained representations that may overlook core functionalities [162]. Program repair models, based on neural machine translation models, have shown slow learning curves, achieving a mere 4.5% repair prediction accuracy even after 10 epochs [27]. This indicates that the structural design of the model hinders its ability to learn and adapt efficiently.

Deployment and Maintenance Phase. In real-world practice, many constraints, such as latency, device memory, and computational costs, need to be satisfied. On the one hand, it prevents some powerful models from being deployed. For instance, Feng et al. [35] noted multiple malware detection models are infeasible on mobile devices due to limitations in computational power, memory, and energy consumption. Zhang et al. [196] highlighted that even standard CodeBERT requires extensive resources for pre-training and fine-tuning before use, which may not be affordable for individual developers. On the other hand, the performance of deployed systems may be compromised as a tradeoff. For example, to be deployed on the user devices, the models have to be pruned or distilled, where considerable performance degradation can be observed [140, 141]. Furthermore, Ganesh et al. [39] showed that while quantization and unstructured pruning can reduce model size, these techniques alone do not improve runtime inference speed or memory usage during execution, unless paired with specialized hardware or processing libraries. Therefore, while constraints exist in real-world deployment that compels simpler models, such compromises often degrade model performance, highlighting the need for techniques that can provide comparable accuracy with greater efficiency.

In summary, the efficacy of LM4Code systems faces significant challenges across data collection, system design, and deployment phases. While researchers continue to advance model capabilities, addressing these fundamental limitations requires careful consideration of the tradeoffs between model performance and practical constraints. Future work needs to focus on developing solutions that can maintain high performance while meeting real-world deployment requirements.

7.3 Compromised Reproducibility

The reproducibility of research findings through independent verification is a fundamental requirement of the scientific process that allows reported improvements to be validated and incrementally built upon [60]. However, our analysis reveals that performance evaluation in LM4Code research may lack sufficient reproducibility. The proposed approaches frequently bypass consistent evaluation protocols, datasets, and implementation details, making it difficult for others to replicate the experiments described in the literature and validate their accuracy [189]. Furthermore, inconsistent performance across different runs or implementations obscures the reliability of findings, as initial results that appear promising frequently fail to fully replicate in subsequent studies [189]. This phenomenon indicates that minor variances in experimental conditions, which are rarely comprehensively reported, can significantly influence outcomes. Limited methodological transparency through selective reporting of details further inhibits reproducibility [67].

7.4 Misleading Benchmarks

Misleading benchmarks and exaggerated claims lead research communities astray and substantially hinder actual progress [156]. The decrease in performance on existing benchmarks can range from 19.6% to 47.7%, leading to drastic ranking changes amongst LLMs [174]. For instance, evaluating models on limited datasets or with improper metrics often amplifies perceived improvements beyond actual capabilities. However, identifying possible pitfalls remains an open challenge for future LM4Code researchers to address. As Liu et al. [94] highlighted, the lack of appropriate baselines

frequently causes considerable gaps between reported and real-world performance. Promoting such flawed evaluations through publications and conferences propagates unreliable techniques built on shaky foundations. This questionable practice squanders precious community resources as researchers may pursue exaggerated claims rather than meaningful progress. More alarmingly, it obscures the path forward for meaningful innovation that solves real-world needs, as progress is measured on inflated claims instead [156]. Ultimately, systemic misleading benchmarks threaten to steer LM4Code research astray, leading the community away from impactful advancements. Establishing rigorous and peer-validated standards for benchmarking to guide productive research directions is an urgent need.

7.5 Security Concerns

The deployment of LM4Code systems introduces significant security vulnerabilities that can compromise both model outputs and intellectual property. These security concerns manifest in two critical areas.

Untrustworthy Results. Both the pitfalls from external attackers and internal drawbacks can lead to untrustworthy outputs, requiring the developers to carefully review and test the generated code. External attacks like evasion, poisoning, and backdoor attacks can induce LM4Code systems to demonstrate manipulated behavior, i.e., generating outputs specifically chosen by attackers. For example, Schuster et al. [132] demonstrate that code completion models can be manipulated to use unsafe encryption algorithms and deprecated security protocol so that the system built based on these suggestions is vulnerable to further attacks. Aghakhani et al. [2] reveal that the backdoors in code generation models can inject various insecure code, including Cross-Site Scripting (CWE-79), Path Traversal (CWE-22), and Deserialization of Untrusted Data (CWE-502). Wan et al. [158] can mislead the code search models to retrieve the code snippets containing malicious actions, such as deleting specific files. Attackers can also provoke systems into producing unexpected predictions so that the application behaves in an unintended way. For example, the adversarial attacks [182, 195] against LM4Code systems can lead to completely wrong predictions. Even without external attacks, the model itself may also naturally generate untrustworthy results, which has been largely reported in various studies. For instance, Pearce et al. [119] revealed vulnerabilities frequently present in GitHub Copilot's generated code. Furthermore, Lukas et al. [99] suggested LMs may inadvertently retain or even leak portions of their training data, which raises concerns regarding unintentional memorization of sensitive information. Niu et al. [111] investigated code completion models like the popular Codex model leaking private information (e.g., passwords and addresses) through generated outputs.

Copyright Infringement. Pitfalls in LM4Code deployment raise significant concerns regarding copyright infringement and loss of intellectual property. Attackers might resort to model stealing, effectively replicating the functionality of proprietary models without authorization [66, 82]. Li et al. [82] demonstrate that imitation models can even exceed the performance of the victim model. Additionally, when models are deployed on user clients, they face the potential threat of reverse engineering. Zhou et al. [198] highlight that the on-device models may leak their confidential information, such as hyperparameters and weights. The risk of copyright infringement and intellectual property theft not only undermines incentives to develop innovative models but also threatens the commercialization prospects and trustworthiness of the LM4Code industry.

These security vulnerabilities, encompassing both untrustworthy outputs and intellectual property risks, pose significant challenges to the widespread adoption of LM4Code systems. The combination of external attacks, internal model limitations, and copyright concerns creates a complex security landscape that requires careful attention. Addressing these challenges requires developing

robust defense mechanisms and establishing clear guidelines for secure model deployment while protecting intellectual property rights.

8 Discussion

8.1 Recommendations for LM4Code Research

Our SLR reveals numerous pitfalls that can undermine the realistic performance and real-world effectiveness of LM4Code systems. These pitfalls span the data, models, evaluation, and deployment phases of the LM4Code lifecycle. LM4Code has become an increasingly prominent research area, evidenced by the rapid increase in publications. A prior survey by Hou et al. [54] uncovered 229 papers on LLMs for SE between 2020 and 2023, while Wang et al. [166] uncovered 350 papers on deep learning for SE between 2015 and 2020. However, our focused study on LM4Code pitfalls only identified 121 relevant papers. This indicates that research attention to pitfalls in LM4Code is still insufficient, compared to the overall research volume. Thus, future LM4Code research must not overlook the pitfalls when applying LM4Code models to SE tasks.

Recognizing Existing Pitfalls. In this study, we summarized multiple common pitfalls associated with LM4Code. These pitfalls, each with distinct implications, highlight the inherent complexities in applying LM4Code to real-world SE problems. As our review results demonstrate, pitfalls can introduce unrealistic performance evaluation, compromise model efficacy, and raise security concerns. Thus, it becomes important for future LM4Code research to recognize and avoid potential pitfalls when building LM4Code systems for SE tasks. To ensure trustworthy findings of LM4Code research, it is essential to demonstrate effectiveness through a rigorous and reliable experimental design that reflects real-world scenarios. Furthermore, although our review has summarized common pitfalls, as previous surveys like Hou et al. [54] present, there exist more than 50 specific LLMs tailored to over 55 SE scenarios. So while we report general implications in some common settings using prevalent models, further investigation is required to discern more specific implications in specific or unconventional scenarios.

Addressing Existing Pitfalls. Addressing the identified pitfalls is vital for advancing robust and reliable LM4Code techniques. As our study reveals, solutions like data cleaning, model explainability, optimized model design, and rigorous benchmarking have shown promise in mitigating certain pitfalls. However, these solutions, although effective in specific contexts, may not be universally applicable due to the complexity and ever-evolving nature of the SE landscape. A coordinated effort by the community is required to establish guidelines and best practices that enable mitigating pitfalls in data construction, model design, performance evaluation, and deployment.

Uncovering New Pitfalls. The dynamic nature of the LM4Code field means novel pitfalls will likely emerge as techniques rapidly evolve. Specifically, the ever-evolving SE landscape, increasingly complex codebases, and new LM4Code techniques provide fertile ground for novel pitfalls to emerge. Thus, identifying emerging pitfalls is critical. The community needs to continuously analyze model reasoning, behaviors, and performance under realistic experimental settings to uncover new pitfalls in a timely manner. For example, testing on diverse unexplored situations or probing model decisions via XAI techniques yields valuable insights. Through periodically updating benchmarks, refining evaluation approaches, and incorporating real-world deployment scenarios, we can ensure that we not only keep pace with the ever-evolving landscape, but also recognize and mitigate new challenges.

8.2 Open Challenges and Research Directions

8.2.1 Improving Data Quality for LM4Code. The processes of data collection and labeling play critical roles in the model construction, determining their efficiency, trustworthiness, and overall

performance [96]. With the emergence of LLMs such as GPT-4 [117], the reliance on massive data has increased significantly. However, as we have reviewed, there are numerous obstacles and unanswered questions associated with these phases.

Volume vs. Quality. The increasing number and widespread use of LLMs such as GPT-4, underscore the inherent conflict between the quantity and quality of data. These models, with their vast number of parameters, heavily rely on extensive datasets to achieve their remarkable performance. For example, Codex [19], which is a variant of the GPT-3.5 framework introduced in 2021, conducted training using a dataset that was sourced from 54 million publicly available software repositories on the GitHub platform, resulting in a total data size of 159 GB. Yet, collecting vast amounts of data, a process that is both time-consuming and labor-intensive, is not a “silver bullet”; data quality is also important. Even with advanced large models, “garbage in–garbage out” still applies [92, 119]. Notably, even widely used benchmark datasets like CodeSearchNet [58] have been observed to contain considerable noise and errors, as highlighted in prior research [142, 147]. Over-reliance on volume can lead to models that are prone to biases, noise, and even adversarial attacks. On the contrary, an excessive focus on data quality can inadvertently decrease the diversity and richness of the dataset. Finding the ideal compromise between these conflicting requirements offers an appealing opportunity for future investigation.

Automated Data Quality Assurance. As the implications of low-quality data, it is essential to utilize high-quality datasets to develop LM4Code approaches. In the dynamic evolution environment of software and LMs, manually searching and examining extensive datasets for data noise or errors is neither practical nor efficient. As a result, there is an urgent need for automated tools and frameworks that can assure and maintain data trustworthiness and quality, particularly for LM4Code models. We can systematically discover and correct data noise, labeling errors, and other anomalies, allowing models to train on robust and reliable datasets and ensuring their reliability and efficiency in real-world applications.

Reasonable Data Distribution. LLMs are probabilistic in nature, and an unreasonable data distribution can lead to flaws within these models. For instance, Cao et al. [13] have pointed out that the data distribution in existing test benchmarks is unbalanced, including disparities in programming languages, imbalances in code granularity, and a lack of advanced features. They found that 95.8% of the benchmarks involved Python, while only five benchmarks involved Java, leading to an insufficient understanding of LLMs’ ability to generate Java code. From a token-level perspective, an unbalanced data distribution can lead to different predictions, which is also one of the significant causes of many attacks, such as adversarial attack and jailbreak. Additionally, the tradeoff between domain knowledge and generalizability in datasets is also a problem worth studying.

8.2.2 Strengthening Robustness and Trustworthiness in LM4Code. LM4Code models are becoming increasingly integrated into the software development lifecycle, influencing everything from code generation to vulnerability detection. Ensuring that these models are robust and trustworthy is essential. This does not just relate to their prediction accuracy but extends to the reliability, interpretability, and generalization capacity of the model, especially in diverse and evolving coding environments [110, 166].

Building Interpretable LM4Code. The black-box nature of LMs has been a long-term concern, especially when LM4Code applications directly influence software development outcomes [22, 150, 166, 170]. Transparency in LM4Code requires an in-depth examination of the correlations and reasoning processes that models depend on, instead of just knowing the model’s predictions. Our review results show that pitfalls can exist throughout the entire LM4Code lifecycle, potentially resulting in spurious correlations. These misleading correlations are based on wrong artifacts for generating predictions, presenting significant challenges for practical real-world applications.

Although prior studies [23, 139, 159, 201] have introduced various XAI techniques into LM4Code, the current solutions lag behind the rapid evolution of LMs [100, 200]. Existing XAI techniques for LMs, in particular, only provide explanations as either the contribution of individual words to the decision or the layer/neuron at which syntax or semantics are encoded [200]. Although helpful, these explanations only offer a fragmented picture of the model's decision-making process, ignoring a considerable amount of its intricate reasoning. As the complexity of LMs grows, there is an imperative need to develop more comprehensive and accessible XAI approaches for LM4Code.

Improving Robustness against Errors and Threats. Recent literature [111, 122, 146] reveals that state-of-the-art LM4Code models like Codex [19], GPT-3, and StarCoder [76] are susceptible to inadvertent data errors and malicious threats. Such pitfalls not only degrade model performance, but raise concerns about the security and trustworthiness of LM4Code systems. Improving model robustness is therefore an urgent need to enable reliable LM4Code adoption. While prior studies [144, 146, 147, 193] have proven that techniques like adversarial training and data augmentation can enhance robustness, more efforts should be spent by our research community to holistically defend against newly emerging issues and threats. Specifically, possible solutions like domain-specific preprocessing and learning, continuous evaluation, hybrid models, and anomaly detection should be explored. Overall, a multilayered defense-in-depth strategy is essential to ensure LM4Code reliability and trustworthiness against growing pitfalls or issues.

Adapting to Ever-Evolving Code Environments. Within the ever-evolving field of software development, new programming languages emerge, old ones get updated, and coding techniques and habits are constantly transforming. Against this backdrop, a primary challenge for LM4Code is ensuring the broad applicability and robust generalizability of LM4Code systems. Prior studies [112, 126, 154, 166] have highlighted the superior generalizability of advanced LMs over traditional ML/DL techniques, particularly when it comes to previously unseen distributions. However, there remain significant research gaps. Specifically, while the improvements in models like the advanced Transformer are promising, they may not always translate to practical efficiency. For instance, Thongtanunam et al. [154] indicate that while the Transformer demonstrates an improvement of 490%–567% on new tokens, its accuracy remains around 10%, indicating a significant room for improvement. Thus, the incorporation of domain-specific knowledge, continuous model updating, and feedback loops with developers could pave the way for more adaptable and reliable LM4Code solutions in an ever-evolving coding landscape.

8.2.3 Towards Reliable Performance Evaluation of LM4Code. Performance evaluation plays a key role in demonstrating the efficiency and capability of the proposed LM4Code approaches. However, as discussed in Section 5, there are several pitfalls that can undermine realistic performance, including inappropriate baselines, inappropriate test sets, reproducibility issues, and inappropriate performance measures. Therefore, it is important to focus future research efforts on solid benchmarks and rigorous evaluation methodology.

Towards Reliable and Standardized Benchmarks. It is important to establish a rigorous evaluation methodology supported by trustworthy and standardized benchmarks. These benchmarks provide consistent frameworks for comparing different LM4Code approaches and set baselines for new techniques [10]. However, our review results reveal that multiple pitfalls related to performance evaluation like inappropriate baselines, limited test sets, and reproducibility issues distort understanding of actual LM4Code capabilities, questioning research correctness [88, 94, 189]. For example, Laaber et al. [72] emphasize the inherent benchmark instability, stressing the importance of identifying and rectifying these instabilities to ensure accurate evaluations. Without addressing pitfalls, perceived model performance can become inflated, leading to potentially misleading conclusions. Related benchmarking challenges have been noted across various research domains like

computer security [156], and cloud computing [24, 133]. Hence, there is an urgent need to prioritize the development of standardized, stable, and reliable benchmarks in LM4Code research. Future research should focus on creating benchmarks that are both comprehensive and representative of real-world scenarios. To achieve this, the community should coordinate on curating representative test sets, establishing strong baselines, quantifying uncertainty, and promoting reproducible experiments, ensuring that advancements in the field of LM4Code are grounded in robust and reliable evaluations. This will not only foster trustworthiness within the research community but also drive meaningful progress in LM4Code.

Towards Reliable Evaluation Metrics for Code. As LMs like GPT-4 grow in complexity, accurately assessing their capabilities and limitations becomes imperative. However, as discussed, traditional evaluation metrics, often borrowed from the NLP domain, may fail to capture the intricacies and details that are crucial for code generation and understanding tasks [128, 131]. For example, metrics like accuracy and BLEU focusing solely on syntactic correctness fail to account for potential semantic errors with practical consequences, as noted by Fan et al. [32]. In addition, most studies rely heavily on automated metrics, but human evaluation remains indispensable for assessing the quality and utility of generated code [26, 118]. Additionally, the variability and uncertainty inherent in generative models like GPT-4 need to be measured and analyzed in order to understand reliability in real-world settings [117]. Therefore, developing holistic and standardized metrics tailored to code intelligence tasks is urgently needed. These should measure syntactic validity, semantic consistency, coherence, human quality judgments, and variability. For LM4Code to move from impressing lab-only evaluation to transforming real-world applications, reliable evaluation metrics are crucial.

8.2.4 Optimizing LM4Code Deployment for Real-World Scenarios. Realizing LM4Code's potential requires addressing deployment challenges in transitioning from controlled research to practice. The integration, security, and scalability of LM4Code techniques become important considerations.

Integrating LM4Code into Developer Workflows and Tools. While modern LM4Code models like GitHub Copilot show potential for integrating AI-driven code suggestions into developers' workflows, realizing this in practice presents unique challenges. First, it is crucial to ensure the correctness and human comprehensibility of suggestions, especially since the generated code can contain errors and vulnerabilities that may be problematic for real-world usage [106, 119, 122]. Additionally, customization and personalization are also important, since every developer has a unique coding style and preferences. Modern LM4Code tools should be adaptable and learn from individual developer behaviors to provide personalized code suggestions. Finally, integration with developer tools like version control and debugging is critical for comprehensive functionality. Future research should focus on addressing these challenges, ensuring that the integration of LM4Code models into developer workflows is smooth, efficient, and beneficial.

Towards LM4Code Security. As discussed in Section 6, LM4Code faces several security concerns that need to be addressed before responsible and ethical deployment can be achieved. These include risks of evasion attacks, data poisoning, privacy leakage, and generated code potentially containing vulnerabilities. Implementing comprehensive solutions is thus critical to safeguard models, data, and users. Advancing the security of LM4Code requires continued research across multiple domains: (1) Adversarial training techniques [199] can potentially increase model robustness against adversarial inputs; (2) Employing formal verification approaches to evaluate the security and correctness of generated code [119]; (3) Enhanced data construction and quality assurance processes are needed to systematically identify and eliminate data poisoning, preventing the propagation of risks [132]; (4) Approaches such as differential privacy [99] and federated learning [77] may strengthen privacy preservation in model training; (5) Watermarking and provenance tracking mechanisms can enable

authentication of model ownership and detect plagiarism or unauthorized use [69, 146]; and (6) Manual examination approaches including code reviews and human-AI collaborative interactions are important. Substantial multidisciplinary efforts are critical to ensure the secure and ethical development and deployment of powerful generative AI systems like LM4Code. This remains an open research challenge requiring continued progress across communities.

Scalability and Latency Concerns. While powerful general LLMs like GPT-4 [117], PaLM [44], and Claude [6] offer public API access, privately deployed specialized LM4Code models can face greater scalability challenges. Developing private models is important to customize LM4Code systems to specific domains and tasks. However, LLMs often comprise billions of parameters, requiring massive computational resources [54]. This poses challenges for real-time efficient integration into developer workflows. Reducing inference latency through methods like knowledge distillation [141] and efficient attention [68] is crucial for reasonable responsiveness. Additionally, scaling up throughput for concurrent users via cloud-native architectures is also essential. Optimizing memory utilization via compression and caching helps in deploying large LM4Code models [141]. Energy-efficient deployment through quantization and lightweight model distillation is likewise critical [46]. Tackling these scalability and latency issues will be critical to realize LM4Code's full potential through deployments that smoothly integrate LM4Code into practical developer environments in a sustainable and user-friendly manner.

8.3 Threats to Validity

This SLR was conducted according to the established guidelines [70, 192] to mitigate potential threats to validity. However, there are still certain limitations primarily associated with our search strategy and the data extraction process used for constructing our paper taxonomy.

One primary threat is selection bias, wherein the selection process may miss some relevant research studies. First, one possible cause may be that some search engines may provide some irrelevant studies or overlook some studies. Another reason could be that our keywords don't cover all the relevant research studies. With an emerging field like LM4Code, important ongoing work may not yet be indexed in the primary digital libraries we searched. In addition, as we stated before, pitfalls do not have consistent keywords or terminology, so our manual checking of the pitfalls of LM4Code papers also has the potential for omissions. To minimize this risk, we systematically performed the paper searching across six major digital libraries in computer science, manually searched top venues, iteratively refined search strings based on a QGS approach defined by Zhang et al. [192], and conducted backward/forward snowballing. Moreover, each manuscript that an individual author expressed uncertainty about regarding its including/excluding underwent thorough discussion between the first two authors before making a final decision.

Another threat is internal validity in constructing our taxonomy of LM4Code pitfalls. A significant contribution of this article is developing a taxonomy to categorize and synthesize key pitfalls across the LM4Code field. To mitigate subjectivity in our taxonomy, we adapted a framework from Arp et al. [7] previously applied across computer security, which was collaboratively validated by the first six authors with LM4Code expertise. At the same time, to enhance accuracy, each primary study classification was reviewed by at least three authors, with disagreements resolved through discussion. Despite the fact that multiple evaluators reduce the possibility of bias, subjective factors persist. To improve the integrity of our taxonomy and offer transparency into our workings, all of our collected research studies and their classification are available in our online repository.

9 Conclusion

In this research study, we conducted a comprehensive and rigorous SLR to examine the pitfalls present in LM4Code. We utilized a well-defined SLR approach and finally obtained 121 relevant

research studies from top-tier venues. We first provided a taxonomy and we classified the existing pitfalls in LM4Code based on the various stages of the LM4Code lifecycle, including data collection and labeling, system design and learning, performance evaluation, and deployment and maintenance. For each stage, we provided a thorough review of the relevant studies based on the pitfall types, implications, and existing solutions. Finally, we described the current challenges and discussed the open opportunities that demand more study in this area. We hope that our work will motivate other researchers, making LMs enhanced for code intelligence more reliable and trustworthy, thereby ensuring their effective deployment into real-world applications.

Acknowledgments

We sincerely thank Zhou Yang and Zhensu Sun, PhD students at Singapore Management University, for their invaluable insights, feedback, and assistance in clearly defining the taxonomy of pitfalls in LMs for code. Their contributions greatly helped shape and improve this work.

References

- [1] codeparrot (CodeParrot). 2022. Retrieved from <https://huggingface.co/codeparrot>
- [2] Zohreh Aghababaeian, Manel Abdellatif, Lionel Briand, S. Ramesh, and Mojtaba Bagherzadeh. 2023. Black-box testing of deep neural networks through test case diversity. *IEEE Transactions on Software Engineering* 49, 5 (May 2023), 3182–3204. DOI: <https://doi.org/10.1109/TSE.2023.3243522>
- [3] Toufique Ahmed, Noah Rose Ledesma, and Premkumar Devanbu. 2023. SynShine: Improved fixing of syntax errors. *IEEE Transactions on Software Engineering* 49, 4 (Apr. 2023), 2169–2181. DOI: <https://doi.org/10.1109/TSE.2022.3212635>
- [4] Ali Al-Kaswan, Maliheh Izadi, and Arie Van Deursen. 2024. Traces of memorisation in large language models for code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12.
- [5] Amazon. 2023. Amazon CodeWhisperer. Retrieved from <https://aws.amazon.com/codewhisperer/>
- [6] Anthropic. 2023. Getting Started with Claude. Retrieved from <https://docs.anthropic.com/claude/docs>
- [7] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and don'ts of machine learning in computer security. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security '22)*, 3971–3988.
- [8] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. arXiv:2108.07732. Retrieved from <https://arxiv.org/abs/2108.07732>
- [9] Gareth Ari Aye, Seohyun Kim, and Hongyu Li. 2021. Learning autocompletion from real-world datasets. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 131–139.
- [10] Dirk Beyer, Stefan Löwe, and Philipp Wendler. 2019. Reliable benchmarking: Requirements and solutions. *International Journal on Software Tools for Technology Transfer* 21, 1 (2019), 1–29.
- [11] Battista Biggio and Fabio Roli. 2018. Wild patterns: Ten years after the rise of adversarial machine learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2154–2156.
- [12] Paul E. Black. 2017. Sard: A software assurance reference dataset. In *Anonymous Cybersecurity Innovation Forum* 85.
- [13] Jialun Cao, Zhiyong Chen, Jiarong Wu, Shing-Chi Cheung, and Chang Xu. 2024. JavaBench: A benchmark of object-oriented code generation for evaluating large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 870–882.
- [14] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*, 2633–2650.
- [15] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet. *IEEE Transactions on Software Engineering* 48, 9 (2021), 3280–3296.
- [16] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.
- [17] Angelica Chen, Jérémy Scheurer, Tomasz Korbak, Jon Ander Campos, Jun Shern Chan, Samuel R. Bowman, Kyunghyun Cho, and Ethan Perez. 2023. Improving code generation by training with natural language feedback. arXiv:2303.16749. Retrieved from <https://arxiv.org/abs/2303.16749>

- [18] Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming Jiang. 2022. Towards training reproducible deep learning models. In *Proceedings of the 44th International Conference on Software Engineering*, 2202–2214.
- [19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [20] Nadezhda Chirkova and Sergey Troshin. 2021. Empirical study of transformers for source code. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 703–715.
- [21] Nadezhda Chirkova and Sergey Troshin. 2023. CodeBPE: Investigating subtokenization options for large language model pretraining on source code. arXiv:2308.00683. Retrieved from <https://arxiv.org/abs/2308.00683>
- [22] Jürgen Cito, Satish Chandra, Chakkrit Tantithamthavorn, and Hadi Hemmati. 2023. Expert perspectives on explainability. *IEEE Software* 40, 3 (2023), 84–88. DOI: <https://doi.org/10.1109/MS.2023.3255663>
- [23] Jürgen Cito, Isil Dillig, Seohyun Kim, Vijayaraghavan Murali, and Satish Chandra. 2021. Explaining mispredictions of machine learning models using rule induction. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, 716–727. DOI: <https://doi.org/10.1145/3468264.3468614>
- [24] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, 143–154.
- [25] Roland Croft, Yongzheng Xie, and Muhammad Ali Babar. 2022. Data preparation for software vulnerability prediction: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1044–1063.
- [26] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming Jack Jiang. 2023. GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [27] Yanguibo Ding, Baishakhi Ray, Premkumar Devanbu, and Vincent J. Hellendoorn. 2021. Patching as translation: The data and the metaphor. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20), Virtual Event*. ACM, 275–286. DOI: <https://doi.org/10.1145/3324884.3416587>
- [28] Jinhao Dong, Yiling Lou, Dan Hao, and Lin Tan. 2023. Revisiting learning-based commit message generation. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 794–805.
- [29] Vinicius H. S. Durelli, Rafael S. Durelli, Simone S. Borges, Andre T. Endo, Marcelo M. Eler, Diego R. C. Dias, and Marcelo P. Guimarães. 2019. Machine learning applied to software testing: A systematic mapping study. *IEEE Transactions on Reliability* 68, 3 (2019), 1189–1212.
- [30] Aryaz Eghbali and Michael Pradel. 2022. CrystalBLEU: Precisely and efficiently measuring the similarity of code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–12.
- [31] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. AC/C++ code vulnerability dataset with code changes and CVE summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories*, 508–512.
- [32] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
- [33] Sen Fang, Tao Zhang, Youshuai Tan, He Jiang, Xin Xia, and Xiaobing Sun. 2023. RepresentThemAll: A universal learning representation of bug reports. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 602–614.
- [34] China Computer Federation. 2023. CCF Recommended List of International Conferences and Periodicals. Retrieved from <https://www.ccf.org.cn/en/Bulletin/2019-05-13/663884.shtml>
- [35] Ruitao Feng, Sen Chen, Xiaofei Xie, Guozhu Meng, Shang-Wei Lin, and Yang Liu. 2021. A performance-sensitive malware detection system using deep learning on mobile devices. *IEEE Transactions on Information Forensics and Security* 16 (2021), 1563–1578. DOI: <https://doi.org/10.1109/TIFS.2020.3025436>
- [36] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. arXiv:2002.08155. Retrieved from <https://arxiv.org/abs/2002.08155>
- [37] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 935–947. DOI: <https://doi.org/10.1145/3540250.3549098>
- [38] Michael Fu, Van Nguyen, Chakkrit Kla Tantithamthavorn, Trung Le, and Dinh Phung. 2023. VulExplainer: A transformer-based hierarchical distillation for explaining vulnerability types. *IEEE Transactions on Software Engineering* 49, 10, 4550–4565.

- [39] Prakhar Ganesh, Yao Chen, Xin Lou, Mohammad Ali Khan, Yin Yang, Hassan Sajjad, Preslav Nakov, Deming Chen, and Marianne Winslett. 2021. Compressing large-scale transformer-based models: A case study on BERT. *Transactions of the Association for Computational Linguistics* 9 (2021), 1061–1080.
- [40] Shuzheng Gao, Cuiyun Gao, Chaozheng Wang, Jun Sun, David Lo, and Yue Yu. 2023. Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1933–1945.
- [41] Shuzheng Gao, Wenxin Mao, Cuiyun Gao, Li Li, Xing Hu, Xin Xia, and Michael R. Lyu. 2024. Learning in the wild: Towards leveraging unlabeled data for effectively tuning pre-trained code models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.
- [42] Shuzheng Gao, Hongyu Zhang, Cuiyun Gao, and Chaozheng Wang. 2023. Keeping pace with ever-increasing data: Towards continual learning of code intelligence models. arXiv:2302.03482. Retrieved from <https://arxiv.org/abs/2302.03482>
- [43] GitHub. 2023. GitHub Copilot. Retrieved from <https://copilot.github.com>
- [44] Google. 2023. Build Generative AI Applications with Google. Retrieved from <https://developers.google.com/generativeai/>
- [45] Alejandro Guerra-Manzanares and Hayretin Bahsi. 2022. On the relativity of time: Implications and challenges of data drift on long-term effective android malware detection. *Computers & Security* 122 (2022), 102835.
- [46] Qianyu Guo, Sen Chen, Xiaofei Xie, Lei Ma, Qiang Hu, Hongtao Liu, Yang Liu, Jianjun Zhao, and Xiaohong Li. 2019. An empirical study towards characterizing deep learning development and deployment across different frameworks and platforms. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 810–822.
- [47] Manish Gupta and Puneet Agrawal. 2022. Compression of deep learning models for text: A survey. *ACM Transactions on Knowledge Discovery from Data* 16, 4 (2022), 1–55.
- [48] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. 2024. UniTSyn: A large-scale dataset capable of enhancing the prowess of large language models for program testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 1061–1072.
- [49] Yiling He, Yiping Liu, Lei Wu, Ziqi Yang, Kui Ren, and Zhan Qin. 2022. MsDroid: Identifying malicious snippets for android malware detection. *IEEE Transactions on Dependable and Secure Computing* 20, 3 (2022), 2025–2039.
- [50] Yiling He, Jian Lou, Zhan Qin, and Kui Ren. 2023. FINER: Enhancing state-of-the-art classifiers with feature attribution to facilitate security analysis. arXiv:2308.05362. Retrieved from <https://arxiv.org/abs/2308.05362>
- [51] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas Reps. 2022. Semantic robustness of models of source code. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 526–537. DOI: <https://doi.org/10.1109/SANER53432.2022.00070>
- [52] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's not a bug, it's a feature: How misclassification impacts bug prediction. In *Proceedings of the 2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 392–401.
- [53] Thong Hoang, Hong Jin Kang, David Lo, and Julia Lawall. 2020. CC2Vec: Distributed representations of code changes. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 518–529.
- [54] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [55] Haonan Hu, Yue Liu, Yanjie Zhao, Yonghui Liu, Xiaoyu Sun, Chakkrit Tantithamthavorn, and Li Li. 2023. Detecting temporal inconsistency in biased datasets for android malware detection. In *Proceedings of the 6th International Workshop on Advances in Mobile App Analysis (A-Mobile)*.
- [56] Yujin Huang, Han Hu, and Chunyang Chen. 2021. Robustness of on-device models: Adversarial attack to deep learning models on android apps. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 101–110.
- [57] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. 2024. Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2515–2537.
- [58] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. Retrieved from <https://arxiv.org/abs/1909.09436>
- [59] Matthew Hutson. 2018. Artificial intelligence faces reproducibility crisis. *Science* 359 (2018), 725–726. DOI: 10.1126/science.359.6377.725
- [60] Peter Ivie and Douglas Thain. 2019. Reproducibility in scientific computing. *ACM Computing Surveys* 51, 3 (2018), 1–36.
- [61] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*, 1219–1231.

- [62] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the 45th International Conference on Software Engineering*.
- [63] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *Proceedings of the 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 135–146.
- [64] Wenxin Jiang, Nicholas Synovic, Matt Hyatt, Taylor R. Schorlemmer, Rohan Sethi, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2023. An Empirical Study of Pre-Trained Model Reuse in the Hugging Face Deep Learning Model Registry. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2463–2475. DOI: <https://doi.org/10.1109/ICSE48619.2023.00206>
- [65] Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, and John C. Grundy. 2021. Practitioners' perceptions of the goals and visual explanations of defect prediction models. In *Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR '21)*. IEEE, 432–443. DOI: <https://doi.org/10.1109/MSR52588.2021.00055>
- [66] Daniel D. Johnson, Daniel Tarlow, and Christian Walder. 2023. RU-SURE? Uncertainty-aware code suggestions by maximizing utility across random user intents. arXiv:2303.00732. Retrieved from <https://arxiv.org/abs/2303.00732>
- [67] Magne Jørgensen, Tore Dybå, Knut Liestøl, and Dag I. K. Sjøberg. 2016. Incorrect results in software engineering experiments: How to improve research practices. *Journal of Systems and Software* 116 (2016), 133–145.
- [68] Jean Kaddour, Joshua Harris, Maximilian Mozes, Herbie Bradley, Roberta Raileanu, and Robert McHardy. 2023. Challenges and applications of large language models. arXiv:2307.10169. Retrieved from <https://arxiv.org/abs/2307.10169>
- [69] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. arXiv:2301.10226. Retrieved from <https://arxiv.org/abs/2301.10226>
- [70] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing systematic literature reviews in software engineering, 2 (Jan. 2007).
- [71] Barbara Kitchenham, Lech Madeyski, and David Budgen. 2023. SEGRESS: Software engineering guidelines for REporting secondary studies. *IEEE Transactions on Software Engineering* 49, 3 (Mar. 2023), 1273–1298. DOI: <https://doi.org/10.1109/TSE.2022.3174092>
- [72] Christoph Laaber, Mikael Basmaci, and Pasquale Salza. 2021. Predicting unstable software benchmarks using static source code features. *Empirical Software Engineering* 26, 6 (2021), 114.
- [73] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Yih, Daniel Fried, Si Yi Wang, and Tao Yu. 2022. DS-1000: A natural and reliable benchmark for data science code generation. arXiv:2211.11501. Retrieved from <https://api.semanticscholar.org/CorpusID:253734939>
- [74] Bo Li, Peng Qi, Bo Liu, Shuai Di, Jingen Liu, Jiquan Pei, Jinfeng Yi, and Bowen Zhou. 2023. Trustworthy AI: From principles to practices. *ACM Computing Surveys* 55, 9 (2023), 1–46.
- [75] Jia Li, Zhuo Li, HuangZhao Zhang, Ge Li, Zhi Jin, Xing Hu, and Xin Xia. 2024. Poison attack and poison detection on deep source code processing models. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–31.
- [76] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from <https://arxiv.org/abs/2305.06161>
- [77] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* 37, 3 (2020), 50–60.
- [78] Yanzhou Li, Shangqing Liu, Kangjie Chen, Xiaofei Xie, Tianwei Zhang, and Yang Liu. 2023. Multi-target backdoor attacks for code pre-trained models. arXiv:2306.08350. Retrieved from <https://arxiv.org/abs/2306.08350>
- [79] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, 292–303. DOI: <https://doi.org/10.1145/3468264.3468597>
- [80] Zhong Li, Minxue Pan, Yu Pei, Tian Zhang, Linzhang Wang, and Xuandong Li. 2022. Robust learning of deep predictive models from noisy and imbalanced software engineering datasets. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.
- [81] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. CCTEST: Testing and repairing code completion systems. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1238–1250.
- [82] Zongjie Li, Chaozheng Wang, Pingchuan Ma, Chaowei Liu, Shuai Wang, Daoyuan Wu, and Cuiyun Gao. 2023. On the feasibility of specialized ability stealing for large language code models. arXiv:2303.03012. Retrieved from <https://arxiv.org/abs/2303.03012>
- [83] Yalan Lin, Chengcheng Wan, Shuwen Bai, and Xiaodong Gu. 2024. VarGAN: Adversarial learning of variable semantic representations. *IEEE Transactions on Software Engineering* (2024) 50, 6 (2024), 1505–1517.

- [84] Zehao Lin, Guodun Li, Jingfeng Zhang, Yue Deng, Xiangji Zeng, Yin Zhang, and Yao Wan. 2022. XCODE: Towards cross-language code representation with large-scale pre-training. *ACM Transactions on Software Engineering and Methodology* 31, 3 (2022), 1–44.
- [85] Wang Ling, Phil Blunsom, Edward Grefenstette, Karl Moritz Hermann, Tomáš Kočiský, Fumin Wang, and Andrew Senior. 2016. Latent predictor networks for code generation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 599–609.
- [86] Chao Liu, Cuiyun Gao, Xin Xia, David Lo, John Grundy, and Xiaohu Yang. 2021. On the reproducibility and replicability of deep learning in software engineering. *ACM Transactions on Software Engineering and Methodology* 31, 1, Article 15 (Oct. 2021), 46 pages. DOI: <https://doi.org/10.1145/3477535>
- [87] Chenyao Liu, Zeqi Lin, Jian-Guang Lou, Lijie Wen, and Dongmei Zhang. 2021. Can neural clone detection generalize to unseen functionalities. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 617–629. DOI: <https://doi.org/10.1109/ASE51524.2021.9678907>
- [88] Hui Liu, Mingzhu Shen, Jiaqi Zhu, Nan Niu, Ge Li, and Lu Zhang. 2022. Deep learning based program generation from requirements text: Are we there yet? *IEEE Transactions on Software Engineering* 48, 4 (Apr. 2022), 1268–1289. DOI: <https://doi.org/10.1109/TSE.2020.3018481>
- [89] Jiawei Liu, ChunqiuSteven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [90] Shigang Liu, GuanJun Lin, Lizhen Qu, Jun Zhang, Olivier De Vel, Paul Montague, and Yang Xiang. 2022. CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation. *IEEE Transactions on Dependable and Secure Computing* 19, 1 (Jan. 2022), 438–451. DOI: <https://doi.org/10.1109/TDSC.2020.2984505>
- [91] Shangqing Liu, Bozhi Wu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2023. ContraBERT: Enhancing code pre-trained models via contrastive learning. arXiv:2301.09072. Retrieved from <https://arxiv.org/abs/2301.09072>
- [92] Yue Liu, Thanh Le-Cong, Ratnadira Widayarsi, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2023. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. arXiv:2307.12596. Retrieved from <https://arxiv.org/abs/2307.12596>
- [93] Yue Liu, Chakkrit Tantithamthavorn, Li Li, and Yepang Liu. 2022. Deep learning for android malware defenses: A systematic literature review. *ACM Computing Surveys* 55, 8 (2022), 1–36.
- [94] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards more realistic evaluation for neural test oracle generation. arXiv:2305.17047. Retrieved from <https://arxiv.org/abs/2305.17047>
- [95] Zhongxin Liu, Xin Xia, Ahmed E. Hassan, David Lo, Zhenchang Xing, and Xinyu Wang. 2018. Neural-machine-translation-based commit message generation: How far are we? In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 373–384.
- [96] David Lo. 2023. Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. arXiv:2309.04142. Retrieved from <https://arxiv.org/abs/2309.04142>
- [97] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning (practical experience report). arXiv:2308.11148. Retrieved from <https://arxiv.org/abs/2308.11148>
- [98] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.
- [99] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2023. Analyzing leakage of personally identifiable information in language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 346–363.
- [100] Andreas Madsen, Siva Reddy, and Sarath Chandar. 2022. Post-hoc interpretability for neural NLP: A survey. *ACM Computing Surveys* 55, 8 (2022), 1–42.
- [101] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the robustness of code generation techniques: An empirical study on GitHub Copilot. arXiv:2302.00438. Retrieved from <https://arxiv.org/abs/2302.00438>
- [102] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, Xudong Liu, and Chunming Hu. 2023. Template-based neural program repair. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1456–1468.
- [103] Bonan Min, Hayley Ross, Elinor Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2021. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys* 56, 2 (2021), 1–40.

- [104] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2022. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. arXiv:2210.14306. Retrieved from <https://api.semanticscholar.org/CorpusID:253117056>
- [105] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2023. When to show a suggestion? Integrating human feedback in AI-assisted programming. arXiv:2306.04930. Retrieved from <https://api.semanticscholar.org/CorpusID:259108906>
- [106] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub Copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*, 1–5.
- [107] Xu Nie, Ningke Li, Kailong Wang, Shangguang Wang, Xiapu Luo, and Haoyu Wang. 2023. Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 52–63.
- [108] Georgios Nikitopoulos, Konstantina Dritsa, Panos Louridas, and Dimitris Mitropoulos. 2021. CrossVul: A cross-language vulnerability dataset with commit data. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1565–1569.
- [109] Changan Niu, Chuanyi Li, Vincent Ng, Dongxiao Chen, Jidong Ge, and Bin Luo. 2023. An empirical comparison of pre-trained models of source code. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2136–2148 (2023). DOI: <https://doi.org/10.1109/ICSE48619.2023.00180>
- [110] Changan Niu, Chuanyi Li, Vincent Ng, and Bin Luo. 2023. CrossCodeBench: Benchmarking cross-task generalization of source code models. arXiv:2302.04030. Retrieved from <https://arxiv.org/abs/2302.04030>
- [111] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy leaks from code generation language models in GitHub Copilot. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*, 2133–2150.
- [112] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. 2022. Generating realistic vulnerabilities via neural code editing: An empirical study. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 1097–1109. DOI: <https://doi.org/10.1145/3540250.3549128>
- [113] Curtis Northcutt, Lu Jiang, and Isaac Chuang. 2021. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research* 70 (2021), 1373–1411.
- [114] Yusuke Oda, Hiroyuki Fudaba, Graham Neubig, Hideaki Hata, Sakriani Sakti, Tomoki Toda, and Satoshi Nakamura. 2015. Learning to generate pseudo-code from source code using statistical machine translation. In *Proceedings of the 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 574–584.
- [115] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2023. Demystifying GPT self-repair for code generation. arXiv:2306.09896. Retrieved from <https://arxiv.org/abs/2306.09896>
- [116] Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. 2023. I know what you trained last summer: A survey on stealing machine learning models and defences. *ACM Computing Surveys* 55, 14s (2023), 1–41.
- [117] OpenAI. 2023. GPT-4 technical report. arXiv:2303.08774 Retrieved from <https://arxiv.org/abs/2303.08774>
- [118] Matteo Paltenghi and Michael Pradel. 2021. Thinking like a developer? Comparing the attention of humans with neural models of code. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 867–879. DOI: <https://doi.org/10.1109/ASE51524.2021.9678712>
- [119] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.
- [120] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*, 2339–2356. DOI: <https://doi.org/10.1109/SP46215.2023.10179324>
- [121] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. TESSERACT: eliminating experimental bias in malware classification across space and time. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security '19)*, 729–746.
- [122] Xutan Peng, Yipeng Zhang, Jingfeng Yang, and Mark Stevenson. 2022. On the security vulnerabilities of text-to-SQL models. arXiv:2211.15363. Retrieved from <https://arxiv.org/abs/2211.15363>
- [123] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable code generation from pre-trained language models. arXiv:2201.11227. Retrieved from <https://arxiv.org/abs/2201.11227>
- [124] Antonio Polino, Razvan Pascanu, and Dan Alistarh. 2018. Model compression via distillation and quantization. In *International Conference on Learning Representations*.
- [125] Mikko Raatikainen, Juha Tiuhonen, and Tomi Männistö. 2019. Software product lines and variability modeling: A tertiary study. *Journal of Systems and Software* 149 (2019), 485–510.

- [126] Md Rafiqul Islam Rabin, Nghi D. Q. Bui, Ke Wang, Yijun Yu, Lingxiao Jiang, and Mohammad Amin Alipour. 2021. On the generalizability of neural program models with respect to semantic-preserving program transformations. *Information and Software Technology* 135 (2021), 106552.
- [127] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. 2024. Towards causal deep learning for vulnerability detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–11.
- [128] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis. arXiv:2009.10297. Retrieved from <https://arxiv.org/abs/2009.10297>
- [129] The Computing Research and Education Association of Australasia. 2023. CORE Rankings Portal. Retrieved from <https://www.core.edu.au/conference-portal>
- [130] Devjeet Roy, Sarah Fakhoury, and Venera Arnaoudova. 2021. Reassessing automatic evaluation metrics for code summarization tasks. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1105–1116.
- [131] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2017. Automatically assessing code understandability: How far are we? In *Proceedings of the 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 417–427.
- [132] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. 2021. You autocomplete me: Poisoning vulnerabilities in neural code completion. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*, 1559–1575.
- [133] Malte Schwarzkopf, Derek G. Murray, and Steven Hand. 2012. The seven deadly sins of cloud computing research. In *Proceedings of the 4th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '12)*.
- [134] Nima Shahbazi, Yin Lin, Abolfazl Asudeh, and H. V. Jagadish. 2023. Representation bias in data: A survey on identification and resolution techniques. *ACM Computing Surveys* 55, 13s (2023), 1–39.
- [135] Changjie Shao, Gaolei Li, Jun Wu, and Xi Zheng. 2024. Exploring semantic redundancy using backdoor triggers: A complementary insight into the challenges facing DNN-based software vulnerability detection. *ACM Transactions on Software Engineering and Methodology* 33, 4 (2024), 1–28.
- [136] Xinyu She, Yanjie Zhao, and Haoyu Wang. 2024. WaDec: Decompiling WebAssembly using large language model. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 481–492.
- [137] Sijie Shen, Xiang Zhu, Yihong Dong, Qizhi Guo, Yankun Zhen, and Ge Li. 2022. Incorporating domain knowledge through task augmentation for front-end JavaScript code generation. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, 1533–1543. DOI : <https://doi.org/10.1145/3540250.3558965>
- [138] Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022. On the evaluation of neural code summarization. In *Proceedings of the 44th International Conference on Software Engineering*, 1597–1608.
- [139] Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. arXiv:2304.05216. Retrieved from <https://arxiv.org/abs/2304.05216>
- [140] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. 2023. Smaller, faster, greener: Compressing pre-trained code models via surrogate-assisted optimization. arXiv:2309.04076. Retrieved from <https://arxiv.org/abs/2309.04076>
- [141] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2022. Compressing pre-trained models of code into 3 MB. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–12.
- [142] Lin Shi, Fangwen Mu, Xiao Chen, Song Wang, Junjie Wang, Ye Yang, Ge Li, Xin Xia, and Qing Wang. 2022. Are we building on the rock? On the importance of data preprocessing for code summarization. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 107–119.
- [143] Benjamin Steenhoeck, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2237–2248.
- [144] Weisong Sun, Yuchen Chen, Guanhong Tao, Chunrong Fang, Xiangyu Zhang, Quanjin Zhang, and Bin Luo. 2023. Backdooring neural code search. arXiv:2305.17506. Retrieved from <https://arxiv.org/abs/2305.17506>
- [145] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. CodeMark: Imperceptible watermarking for code datasets against neural code completion models. arXiv:2308.14401. Retrieved from <https://arxiv.org/abs/2308.14401>
- [146] Zhensu Sun, Xiaoning Du, Fu Song, Mingze Ni, and Li Li. 2022. CoProtector: Protect open-source code against unauthorized training usage with data poisoning. In *Proceedings of the ACM Web Conference 2022*, 652–660.

- [147] Zhensu Sun, Li Li, Yan Liu, Xiaoning Du, and Li Li. 2022. On the importance of building high-quality training datasets for neural code search. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 1609–1620. DOI : <https://doi.org/10.1145/3510003.3510160>
- [148] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1433–1443.
- [149] Alexey Svyatkovskiy, Sebastian Lee, Anna Hadjitofi, Maik Riechert, Juliana Vicente Franco, and Miltiadis Allamanis. 2021. Fast and memory-efficient neural code completion. *IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 329–340.
- [150] Chakkrit Tantithamthavorn, Jürgen Cito, Hadi Hemmati, and Satish Chandra. 2023. Explainable AI for SE: Challenges and future directions. *IEEE Software* 40, 3 (2023), 29–33. DOI : <https://doi.org/10.1109/MS.2023.3246686>
- [151] Chakkrit Tantithamthavorn and Ahmed E. Hassan. 2018. An experience report on defect modelling in practice: Pitfalls and challenges. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP '18)*. Frances Paulisch and Jan Bosch (Eds.), ACM, 286–295. DOI : <https://doi.org/10.1145/3183519.3183547>
- [152] Chakkrit Tantithamthavorn and Jirayus Jiarpakdee. 2021. Explainable AI for software engineering. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. IEEE, 1–2. DOI : <https://doi.org/10.1109/ASE51524.2021.9678580>
- [153] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, Akinori Ihara, and Kenichi Matsumoto. 2015. The impact of mislabelling on the performance and interpretation of defect prediction models. In *Proceedings of the 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 812–823.
- [154] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated code transformation to support modern code review process. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, 237–248. DOI : <https://doi.org/10.1145/3510003.3510067>
- [155] Zhiyi Tian, Lei Cui, Jie Liang, and Shui Yu. 2022. A comprehensive survey on poisoning attacks and countermeasures in machine learning. *ACM Computing Surveys* 55, 8 (2022), 1–35.
- [156] Erik van der Kouwe, Gernot Heiser, Dennis Andriess, Herbert Bos, and Cristiano Giuffrida. 2019. SoK: Benchmarking flaws in systems security. In *Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 310–325.
- [157] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 30.
- [158] Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You see what I want you to see: Poisoning vulnerabilities in neural code search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, 1233–1245. DOI : <https://doi.org/10.1145/3540250.3549153>
- [159] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What do they capture?: A structural analysis of pre-trained language models for source code. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, 2377–2388. DOI : <https://doi.org/10.1145/3510003.3510050>
- [160] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S. Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*. ACM, 397–407. DOI : <https://doi.org/10.1145/3238147.3238206>
- [161] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging pre-trained models and downstream tasks for source code understanding. In *Proceedings of the 44th International Conference on Software Engineering*, 287–298.
- [162] Huaijin Wang, Pingchuan Ma, Yuanyuan Yuan, Zhibo Liu, Shuai Wang, Qiyi Tang, Sen Nie, and Shi Wu. 2023. Enhancing DNN-based binary code function search with low-cost equivalence checking. *IEEE Transactions on Software Engineering* 49, 1 (Jan. 2023), 226–250. DOI : <https://doi.org/10.1109/TSE.2022.3149240>
- [163] Jindong Wang, Xixu Hu, Wenxin Hou, Hao Chen, Runkai Zheng, Yidong Wang, Linyi Yang, Haojun Huang, Wei Ye, Xiubo Geng, et al. 2023. On the robustness of ChatGPT: An adversarial and out-of-distribution perspective. arXiv:2302.12095. Retrieved from <https://arxiv.org/abs/2302.12095>
- [164] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software testing with large language model: Survey, landscape, and vision. arXiv:2307.07221. Retrieved from <https://arxiv.org/abs/2307.07221>
- [165] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
- [166] Simin Wang, Liguang Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2022. Machine/deep learning for software engineering: A systematic literature review. *IEEE*

- Transactions on Software Engineering* 49, 3 (Mar. 2022), 1188–1231. DOI: <https://doi.org/10.1109/TSE.2022.3173346>
- [167] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, et al. 2022. ReCode: Robustness evaluation of code generation models. arXiv:2212.10264. Retrieved from <https://arxiv.org/abs/2212.10264>
 - [168] Wenhua Wang, Yuqun Zhang, Yulei Sui, Yao Wan, Zhou Zhao, Jian Wu, Philip S. Yu, and Guandong Xu. 2022. Reinforcement-learning-guided source code summarization using hierarchical attention. *IEEE Transactions on Software Engineering* 48, 1 (Jan. 2022), 102–119. DOI: <https://doi.org/10.1109/TSE.2020.2979701>
 - [169] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv:2109.00859. Retrieved from <https://arxiv.org/abs/2109.00859>
 - [170] Cody Watson, Nathan Cooper, David Nader Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A systematic literature review on the use of deep learning in software engineering research. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–58.
 - [171] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing large language models with completion engines for automated program repair. arXiv:2309.00608. Retrieved from <https://arxiv.org/abs/2309.00608>
 - [172] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2023. Exploring parameter-efficient fine-tuning techniques for code generation with large language models. arXiv:2308.10462. Retrieved from <https://arxiv.org/abs/2308.10462>
 - [173] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. arXiv:2305.18607. Retrieved from <https://arxiv.org/abs/2305.18607>
 - [174] Chunqiu Steven Xia, Yinlin Deng, and Lingming Zhang. 2024. Top leaderboard ranking = Top coding proficiency, always? EvoEval: Evolving coding benchmarks via LLM. arXiv:2403.19114. Retrieved from <https://arxiv.org/abs/2403.19114>
 - [175] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. ACM.
 - [176] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 1–10.
 - [177] Jiayun Xu, Yingjiu Li, and Robert H. Deng. 2021. Differential training: A generic framework to reduce label noises for android malware detection. In *Proceedings of the Network and Distributed System Security Symposium*. Retrieved from <https://api.semanticscholar.org/CorpusID:231879075>
 - [178] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. 2024. CREF: An LLM-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 882–894.
 - [179] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024. Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond. *ACM Transactions on Knowledge Discovery from Data* 18, 6 (2024), 1–32.
 - [180] Xu Yang, Shaowei Wang, Yi Li, and Shaohua Wang. 2023. Does data sampling improve deep learning-based vulnerability detection? Yeas! and nays! In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2287–2298.
 - [181] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A survey on deep learning for software engineering. *ACM Computing Surveys* 54, 10s (2022), 1–73.
 - [182] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering*, 1482–1493.
 - [183] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy backdoor attack for code models. *IEEE Transactions on Software Engineering* 50, 4 (2024), 721–741.
 - [184] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2023. What do code models memorize? An empirical study on large language models of code. arXiv:2308.09932. Retrieved from <https://arxiv.org/abs/2308.09932>
 - [185] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* 4, 2 (2024), 100211.
 - [186] Suraj Yatish, Jirayus Jiarpakdee, Patanamon Thongtanunam, and Chakkrit Tantithamthavorn. 2019. Mining software defects: Should we consider affected releases? In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 654–665.

- [187] Rahul Yedida and Tim Menzies. 2022. On the value of oversampling for deep learning in software defect prediction. *IEEE Transactions on Software Engineering* 48, 8 (2021), 3103–3116.
- [188] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th International Conference on Mining Software Repositories*, 476–486.
- [189] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '22)*, Virtual. ACM, 39–51. DOI: <https://doi.org/10.1145/3533767.3534390>
- [190] Zhengran Zeng, Yuqun Zhang, Haotian Zhang, and Lingming Zhang. 2021. Deep just-in-time defect prediction: How far are we? In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*. ACM, New York, NY, 427–438. DOI: <https://doi.org/10.1145/3460319.3464819>
- [191] Fengyi Zhang, Bihuan Chen, Yufei Zhao, and Xin Peng. 2023. Slice-based code change representation learning. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 319–330.
- [192] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying relevant studies in software engineering. *Information and Software Technology* 53, 6 (2011), 625–637.
- [193] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. 2022. Towards robustness of deep program processing models detection, estimation, and enhancement. *ACM Transactions on Software Engineering and Methodology* 31, 3, Article 50 (July 2022), 1–40. DOI: <https://doi.org/10.1145/3511887>
- [194] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020. Generating adversarial examples for holding robustness of source code processing models. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 01 (Apr. 2020), 1169–1176. DOI: <https://doi.org/10.1609/aaai.v34i01.5469>
- [195] Weiwei Zhang, Shengjian Guo, Hongyu Zhang, Yulei Sui, Yinxing Xue, and Yun Xu. 2023. Challenging machine learning-based clone detectors via semantic-preserving code transformations. *IEEE Transactions on Software Engineering* 49, 5 (May 2023), 3052–3070. DOI: <https://doi.org/10.1109/TSE.2023.3240118>
- [196] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet code is healthy: Simplifying programs for pre-trained models of code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, 1073–1084. DOI: <https://doi.org/10.1145/3540250.3549094>
- [197] Yunhui Zheng, Saurabh Pujar, Burn Lewis, Luca Buratti, Edward Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. 2021. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 111–120.
- [198] Mingyi Zhou, Xiang Gao, Jing Wu, John Grundy, Xiao Chen, Chunyang Chen, and Li Li. 2023. ModelObfuscator: Obfuscating model information to protect deployed ML-based systems, 1005–1017 (2023). DOI: <https://doi.org/10.1145/3597926.3598113>
- [199] Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald Gall. 2022. Adversarial robustness of deep code comment generation. *ACM Transactions on Software Engineering and Methodology* 31, 4 (July 2022), 1–30. DOI: <https://doi.org/10.1145/3501256>
- [200] Julia El Zini and Mariette Awad. 2022. On the explainability of natural language processing deep models. *ACM Computing Surveys* 55, 5 (2022), 1–31.
- [201] Deqing Zou, Yawei Zhu, Shouhuai Xu, Zhen Li, Hai Jin, and Hengkai Ye. 2021. Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. *ACM Transactions on Software Engineering and Methodology* 30, 2 (Mar. 2021), 1–31. DOI: <https://doi.org/10.1145/3429444>

Appendix

A Details for Paper Collection and Selection

Since we explore the state-of-the-art research of the pitfalls within LM4Code, we first outline our methodology for identifying relevant research studies, and then provide an overview of our collected literature. We follow the rigorous methodology proposed by Kitchenham and Charters [70], Kitchenham et al. [71], and Zhang et al. [192] to perform our lightweight SLR. The primary steps in our SLR can be summarized as follows: (1) planning the review and formulating a review protocol, (2) proposing RQs, (3) designing search strategies and proposing inclusion/exclusion

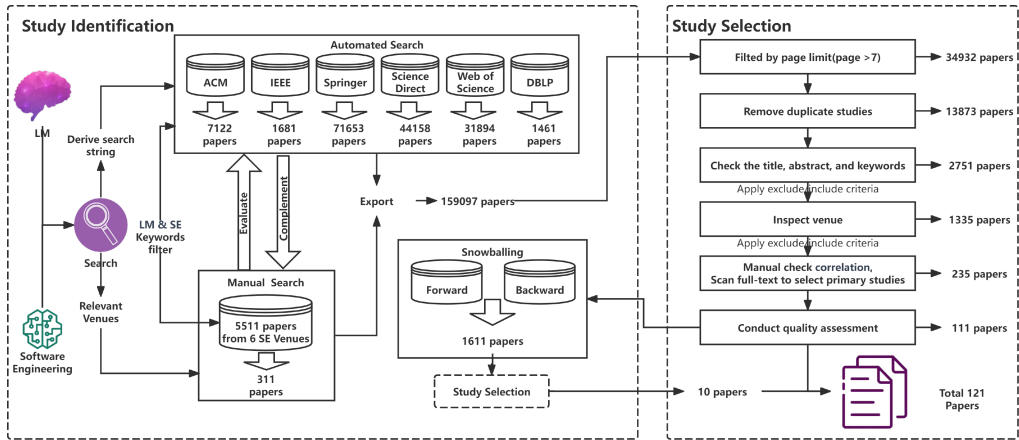


Fig. 12. The overview of the review process.

Table 4. Publication Venues for Manual Search

Acronym	Venues
ASE	International Conference on Automated Software Engineering
ESEC/FSE	Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering
ICSE	International Conference on Software Engineering
ISSTA	International Symposium on Software Testing and Analysis
TOSEM	Transactions on Software Engineering and Methodology
TSE	Transactions on Software Engineering

criteria, (4) conducting a lightweight snowballing, (5) data extraction, and (6) data synthesis. The outline for our review can be represented as presented in Figure A1.

A.1 RQs

- *RQ1: What types of pitfalls are prevalent in LMs for code intelligence?* This RQ aims to identify the prevalent pitfalls in LM4Code systems, exploring how they could affect various stages of the learning-based system lifecycle.
- *RQ2: What solutions have been proposed to address these biases and pitfalls?* This RQ reviews the existing body of literature to pinpoint proposed approaches for mitigating the identified pitfalls.
- *RQ3: What are the implications of these pitfalls?* This RQ investigates the implications of the identified pitfalls, specifically focusing on their impacts on the effectiveness, reliability, and ethical considerations of automated code intelligence systems.

A.2 Search Strategy

To find out all potentially relevant research papers, we utilized the QGS [192] approach, which combines both manual and automated search strategies. Using QGS offers an optimal balance between efficiency and research coverage, as evidenced in several previous studies[29, 166]. As illustrated in Figure A1, our search strategy involved the following sequential steps:

- (1) Select appropriate publication venues for manual search and select digital databases for automated search that encompass all the chosen venues.

- (2) Establish QGS: Screen all papers for manual search and filter by inclusion/exclusion criteria (defined in Table A1).
- (3) Define search string based on domain knowledge from LMs and SE.
- (4) Conduct an automated search using the search keywords defined in Step 3.
- (5) Evaluate the quality of included studies through QGS.

In our research approach, we integrated both manual and keyword-based search methodologies to identify relevant papers. During the manual approach, we focused on six top-tier SE conferences and journals, namely ICSE, ESEC/FSE, ASE, ISSTA, TOSEM, and TSE, all of which are CCF [34] A-ranked venues in the SE domains (as indicated in Table A1). We systematically crawled a list comprising 5,511 published papers from the top venues. After automating scanning with scripts, we thoroughly verified and discovered 311 papers relating to LM4Code. These 311 relevant papers served as the foundation for developing the QGS for the following automated search. Our search string should include two sets of keywords: one for LMs and another for code intelligence. Only if the paper contains both types of keywords does it have a higher probability of being the one we require. The full list of search keywords is as follows:

- Keywords related to LMs: “LLM” OR “Large Language Model*” OR “Language Model*” OR “LM” OR “PLM” OR “Pre-trained” OR “Pre-training” OR “Natural Language Processing” OR “NLP” OR “Machine Learning” OR “ML” OR “Deep Learning” OR “DL” OR “Artificial Intelligence” OR “AI” OR “Transformer” OR “BERT” OR “CODEX” OR “GPT” OR “T5” OR “Sequence Model*” OR “Attention Model*” OR “Transfer Learning” OR “Neural Network*” OR “ChatGPT” OR “GPT-*” OR “Deep neural network*” OR “DNN*”
- Keywords related to code intelligence tasks: “Software Engineering” OR “Software Development” OR “Program*” OR “Software Testing” OR “Software Mainten*” OR “SE” OR “Software Lifecycle” OR “Software Design*” OR “Code representation” OR “Code generation” OR “Code comment generation” OR “Code search” OR “Code localization” OR “Code completion” OR “Code summarization” OR “Method name generation” OR “Bug detection” OR “Bug localization” OR “Vulnerability detection” OR “Testing techniques” OR “Test case generation” OR “Program analysis” OR “Bug classification” OR “Defect prediction” OR “Program repair” OR “Code clone detection” OR “Bug report” OR “Software quality evaluation” OR “SATD detection” OR “Code smell detection” OR “Compiled-related” OR “Code review” OR “Software classification” OR “Code classification” OR “Code change” OR “Incident detection” OR “Requirement extraction” OR “Requirement traceability” OR “Requirement validation” OR “Effort cost prediction” OR “Mining GitHub/GitHub mining” OR “Mining SO (StackOverflow)/SO mining” OR “Mining app/App mining” OR “Mining tag/Tag mining” OR “Developer-based mining”

It’s important to highlight that our list of keywords is specific to LM4Code, and we intentionally omitted keywords that are related to pitfalls during the paper search process. The reason behind this derives from the ambiguity around the term “pitfalls,” which is open to various interpretations. We decided to rely on our rigorous inclusion/exclusion criteria because it is difficult to precisely categorize the types of pitfalls that exist within LM4Code, which is also the main motivation behind this taxonomy study. We were able to include relevant papers with this methodology, even if they didn’t explicitly mention “pitfalls” in their content.

After establishing the search string, we proceeded to conduct an automated search across six widely used databases to ensure comprehensive coverage of all relevant published papers. Specifically, our search spanned four major academic publishers: ACM Digital Library, IEEE Xplorer, Springer, and Science Direct. Additionally, we included two renowned indexing databases: DBLP

Table A2. Inclusion and Exclusion Criteria

Inclusion Criteria	
(I1)	Studies explicitly utilizing LMs.
(I2)	Studies claim that the study involves code-related tasks.
(I3)	Studies highlighting pitfalls, particularly emphasizing unrealistic performance evaluation or factors that negatively influence performance in LM4Code.
Exclusion Criteria	
(E1)	Studies whose full-text is inaccessible.
(E2)	The study whose number of pages is less than 8.
(E3)	Redundant or nearly identical studies from the same authors.
(E4)	Papers not written in English.
(E5)	SLRs, reviews, or surveys.
(E6)	Studies from workshops, doctoral symposiums, books, theses, monographs, keynotes, or panels.
(E7)	Non peer-reviewed academic literature.
(E8)	Studies not related to LMs or code intelligence.
(E9)	Studies emphasizing LM4Code without discussing pitfalls related to realistic performance.
(E10)	Studies with a primary focus on cyberattacks or Operating Systems.
(E11)	Studies that are published in a journal or conference with a CORE [129] ranking of less than A.

and Web of Science, which indexes several other smaller academic databases. Similarly to previous studies [25, 166], we did not use other search engines like Google Scholar due to the existence of excessive irrelevant information in the search results and the requirement for subjective criteria to choose when to stop the search process. Finally, we obtained 7,122 papers from the ACM Digital Library, 1,681 papers from IEEE Xplore, 71,653 papers from Springer, 44,158 papers from ScienceDirect, 44,158 papers from Web of Science, and 1,461 papers from DBLP.

A.3 Study Selection

From among the papers collected by the paper search process, we attempted to select any research study that focused on the pitfalls of LM4Code. As we discussed before, we define these “pitfalls” as any significant issues or constraints present within the datasets, model architectures, experimental designs, or even model deployment that could potentially undermine the reliability or realistic performance of the proposed LM4Code systems. The inclusion/exclusion criteria we adopted, shown in Table A2, were inspired by similar studies [25, 93, 125]. It is noted that to maintain a reliable taxonomy of the pitfalls of LM4Code based on high-quality research studies, we removed the studies published in low-quality venues: venue ranking below A using the CORE ranking system [129].

To determine whether the studies met the inclusion requirements, we combined thorough manual assessment with automated script filtering. The study selection process involves six distinct stages, as illustrated in Figure A1. The first two stages (filtering and deduplication) used automated scripts, substantially reducing the initial set to 13,873 papers. Subsequently, in stages three and four, we applied the inclusion/exclusion criteria, delving into the titles, abstracts, keywords, and publication venues of each paper. It led to a sharp reduction in the number of remaining papers, leaving us with 1,335. The primary reason for exclusion was the lack of keywords correlating with both LMs and code intelligence. Furthermore, we dismissed 890 papers classified as grey literature or misaligned

with our main focus, as well as 105 SLR articles to maintain our emphasis on the pitfalls associated with LM4Code tasks. In the fifth and sixth stages, every paper underwent a manual evaluation for relevance and QA, which led to the final selection of 111 papers.

For SLRs, it is important to analyze the quality of collected studies to ensure that we form an accurate and unbiased representation of the actual research [70]. Thus, we undertook the QA process using a pre-defined quality checklist. We established four quality assurance criteria (QA1–QA4) to reassess all selected papers.

- QA1. Are the pitfalls of LM4Code clearly described?
- QA2. Are the implications of the LM4Code pitfalls clearly stated or demonstrated?
- QA3. Is there a robust evaluation or solution for the identified LM4Code pitfalls in the proposed methodology?
- QA4. Is the contribution of the research clearly stated?

We critically evaluate and rate each QA on a scale of 0–4 (with 4 denoting “high,” while 0 denoting “low”). Then, we calculate the average quality score based on four quality criteria. We set the threshold equal to 2.5 (50% of the percentage score), which means if the average quality score of the research isn’t larger than 2.5, the study would be excluded.

A.4 Snowballing

In order not to miss some important work, we conducted a lightweight snowballing [], which is a commonly used search approach to complement automated queries. We executed both forward and backward snowballing, incorporating references and citations into consideration. Specifically, each primary study was examined for its references (backward snowballing) and its subsequent citations using Google Scholar (forward snowballing). The set of 111 papers from the prior step served as the initial set. From the forward and backward snowballing, we garnered 1,003 and 1,819 papers, respectively. Following the processes of merging, deduplication, and the removal of articles already uncovered in our automated and manual searching, we ended up with a pool of 1,611 papers. These papers underwent the same study selection process, culminating in the identification of an extra 10 papers. As a result, we collected a total of 121 papers focusing on LM4Code pitfalls.

Received 7 January 2025; revised 8 May 2025; accepted 7 July 2025